



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA

ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ

TABLE STRUCTURE RECOGNITION USING MULTIMODAL TRANSFORMERS

ANALÝZA STRUKTURY TABULEK POMOCÍ MULTIMODÁLNÍCH TRANSFORMERŮ

MASTER'S THESIS

DIPLOMOVÁ PRÁCE

AUTHOR

AUTOR PRÁCE

Bc. VOJTĚCH VLACH

SUPERVISOR

VEDOUCÍ PRÁCE

Ing. MICHAL HRADIŠ, Ph.D.

BRNO 2025

Master's Thesis Assignment



164704

Institut: Department of Computer Graphics and Multimedia (DCGM)
Student: **Vlach Vojtěch, Bc.**
Programme: Information Technology and Artificial Intelligence
Specialization: Machine Learning
Title: **Table Structure Recognition Using Multimodal Transformers**
Category: Image Processing
Academic year: 2024/25

Assignment:

1. Familiarize yourself with multimodal transformer architectures.
2. Review the current state-of-the-art methods for recognizing table structures in documents, emphasizing end-to-end transformer models.
3. Select or propose an appropriate method capable of automatically analyzing table structures from images.
4. Prepare or adapt a dataset suitable for your experiments. You may extend or update an existing dataset.
5. Implement the chosen method and conduct experiments on your prepared dataset.
6. Compare your experimental results with existing methods and discuss potential directions for future improvements.
7. Create a concise video presentation summarizing your research, its objectives, methodologies, and key findings.

Literature:

- Jiawei Wang, Kai Hu, Qiang Huo: DLAFormer: An End-to-End Transformer For Document Layout Analysis. ICDAR, 2024.
- Brandon Smock Rohith Pesala Robin Abraham: PubTables-1M: Towards comprehensive table extraction from unstructured documents. CVPR, 2022

Requirements for the semestral defence:

Points 1-4 and progress in point 5.

Detailed formal requirements can be found at <https://www.fit.vut.cz/study/theses/>

Supervisor: **Hradiš Michal, Ing., Ph.D.**
Head of Department: Černocký Jan, prof. Dr. Ing.
Beginning of work: 1.11.2024
Submission deadline: 21.5.2025
Approval date: 15.5.2025

Abstract

This thesis introduces the topic of Table Structure Recognition (TSR), which is used to analyze and reconstruct scanned tables. Current methods are introduced and expanded upon to create a Table Structure Recognition system. First, the Optical Character Recognition (OCR) system detects and transcribes words. The table structure is created using adjacency matrices representing word relation classes (same cell, column clusters, row clusters). The proposed architecture consists of a CNN backbone, a multimodal decoder transformer, class-wise prediction heads, and post-processing table reconstruction algorithm. The architecture is proven to work and is comparable with reference literature on the *PubTables-1M* dataset. The trained models are also fine-tuned on a custom *HeritageTabNet* dataset with positive improvement on both datasets

Abstrakt

Tato práce se zabývá rozpoznáním struktury tabulek pro analýzu a rekonstrukci tabulek z vyfocených nebo skenovaných dokumentů. Práce představljuje současné metody, rozšiřuje je a jejím výsledkem je systém pro rozpoznání tabulek. Konkrétně je použit systém rozpoznání písma (Optical Character Recognition - OCR) pro detekování a přepis jednotlivých slov. Struktura tabulky je tvořena pomocí matice sousedností reprezentující shlukové vztahy mezi slovy (shluky typu buňka, sloupec, řádek). Představená architektura je tvořena konvolučním předzpracováním, multimodálním transformerem, predikčními hlavami pro každý typ vztahu a algoritmem rekonstrukce tabulky. Architektura je funkční a je porovnatelná s referenční literaturou na datasetu *PubTables-1M*. Natrénované modely jsou také doladěné na novém datasetu *HeritageTabNet* s pozitivní změnou na obou datasetech

Keywords

Table Structure Recognition, Table detection, Document Structure Analysis, Multimodal transformer, Word relation prediction, Optical Character Recognition, Deep learning, Computer vision

Klíčová slova

Rozpoznání struktury tabulky, detekce tabulek, analýza struktury dokumentů, multimodální transformer, predikce sousednosti buněk, optické rozpoznávání písma, hluboké učení, počítačové vidění

Reference

VLACH, Vojtěch. *TABLE STRUCTURE RECOGNITION USING MULTIMODAL TRANSFORMERS*. Brno, 2025. Master's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Michal Hradiš, Ph.D.

TABLE STRUCTURE RECOGNITION USING MULTIMODAL TRANSFORMERS

Declaration

I hereby declare that this Master's thesis was prepared as an original work by the author under the supervision of Ing. Michal Hradiš Ph.D. The supplementary information was provided by Ing. Martin Kostelník, Ing. Jan Kohút, and Ing. Martin Kišš. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis.

.....

Vojtěch Vlach
May 21, 2025

Acknowledgements

I would like to thank my supervisor Ing. Michal Hradiš Ph.D. for providing a creative environment and computational power. Also thanks to my colleagues Ing. Martin Kostelník for his expertise on multimodal transformers and a training script skeleton, Ing. Jak Kohút for providing me with a fresh-off-the-oven OCR model for my experiments, Ing. Martin Kišš for being considerate of my computational needs on shared resources during training, and to the three unnamed student colleagues for annotating the majority of the dataset as a part of the Orbis Pictus project grant. Biggest thanks goes to my wife who supported me tirelessly and only rarely gave up on me and/or the thesis.

Contents

1	Introduction	6
2	Table structure recognition	8
2.1	Datasets	8
2.2	Evaluation metrics	11
2.3	PubTables (2022)	13
2.4	SPRINT (2024)	15
2.5	ClusterTabNet (2024)	16
2.6	Discussion	18
3	Related literature	19
3.1	Detection transformer (2020)	20
3.2	DLAFormer (2024)	20
4	HeritageTabNet dataset	22
4.1	Annotation format	23
4.2	Annotation process	24
4.3	Qualitative baseline analysis	27
5	Proposed method and implementation	29
5.1	Proposed method summary	31
5.2	Word relation prediction ground-truth definitions	32
5.3	Word relation prediction transformer	33
5.4	Table structure reconstruction	34
6	Experiments and results	36
6.1	Training on PubTables-1M	36
6.2	Fine-tuning on HerritageTabNet	38
6.3	Table structure reconstruction with clustering	39
6.4	Discussion	41
7	Conclusion	44
	Bibliography	45
A	Attachments structure	48
B	Table extraction examples	49

List of Figures

2.1	Table Detection (TD) and Table Structure Recognition (TSR) is a sub-field of Document structure analysis, sometimes using Optical Character Recognition systems. The main goal is to detect, understand, and recreate tabular data in scanned documents.	9
2.2	Examples of dataset described in this thesis: Large generated datasets <i>PubTables-1M</i> [20] and <i>PubTabNet</i> [26], and smaller scanned datasets <i>GloSAT</i> [27] and <i>cTDaR</i> [4].	10
2.3	Example of adjacency relations between cells to measure F1 DAR (Directed adjacency relations) [6]. For N cells, a matrix N x N is generated, representing the directed nearest neighbor relation between cells (vertical and horizontal). For example, the cells 0 and 1 are horizontal neighbors, so there is True in the horizontal neighborhood matrix on the position [0, 1]. In total, this table has 12 relations, 12 detected relations, 10 correctly detected relations (dark blue and light blue), and 2 incorrectly detected relations (red).	12
2.4	Example visualization of strong and weak connected components in a table structure with clustering neighbours relations between cells.	12
2.5	Example visualization of column and row bounding box creation for applying object detection metrics to word relation prediction tasks. The bounding boxes are created by getting the minimum and maximum coordinates of all words inside (as seen in <i>ClusterTabNet</i> [18]) with the exception of joined cells, which get counted only in X coordinates for columns (and Y for rows) to prevent stretching multiple boxes to the same one.	14
2.6	This image is taken from <i>PubTables</i> [20], where all three table extraction subtasks are handled with an object detection approach (table detection, structure recognition, and functional analysis). For TSR and FA, the proposed detection model detects objects in 5 classes at once: column and row for TSR, column header , projected row header , and spanning cell for FA. Two model architectures are compared in the paper: Detection Transformer (DETR) [2] and Faster R-CNN [19] with varied results discussed in chapter 2.3.	15
2.7	This image is taken from SPRINT [13], where table logical structure recognition is attended with an Img2Seq model. The table input is downsampled and resized to a square. An encoder-decoder transformer is then used to produce the output sequence of tokens/letters. Output vocabulary is made out of 5 tokens/letters (F, L, U, X, N). F (regular cell), L (cell merged with left neighbor), U (cell merged with upper neighbor), X (cell merged with both upper and left neighbor), N (new line/row of the table). GCA means global context attention encoder described in [14].	16

2.8	This image is taken from ClusterTabNet [18], where table detection and table structure recognition are attended. The architecture relies on a pre-build OCR engine to detect and transcribe words. Text embedding is then made for each word preprocessed with a simple replacement algorithm („The date is 12.9.“ -> „Aaaaaaaa11,1,“) and position embeddings for each coordinate (x0, y0, x1, and y1). A CNN backbone preprocesses the input image cut into crops around words (with padding). A transformer encoder processes all this information, and the last step consists of several parallel similarity-based prediction heads for each class of relation between cells. The result of this architecture is a similarity score for each word pair in every class. For an input of N words and C prediction head/relation classes (same row, column, table), the output is CxNxN.	17
3.1	This image is taken from <i>End-to-end object detection with transformers</i> [2], which introduces a detection transformer architecture (DETR) for object detection in the form of direct set prediction. Image is processed with ResNet backbone, and positional embedding is added. Latent embeddings are then created using a transformer encoder. A transformer decoder then takes object queries (a small fixed number of learned positional embeddings). Output comes through a feed-forward network (FFN) to produce predictions (class and bounding box) or a „no object“ class.	20
3.2	This image is taken from <i>DLAFormer: An End-to-End Transformer For Document Layout Analysis</i> paper [24], which introduces a novel approach to document layout analysis (DLA) and its several subtasks. Clustering text lines into regions and reading order prediction is visible given purple lines (defining a reading order of lines) and green lines (defining a reading order of regions, as well as text lines not belonging to the same region). All this using exclusive relations between graphical elements visible on the right side. T_i are text lines detected by an OCR preprocessor, and G_i are graphical objects proposed by the encoder part without OCR knowledge. All these graphical elements are also classified into logical role classes.	21
4.1	Examples of HerritageTabNet, a new dataset created for this thesis. It consists of 600 tables in PAGE XML format, providing table bounding boxes on a page, cell bounding boxes and logical structure in a table, and text transcriptions.	23
4.2	PAGE XML format (Page Analysis and Ground-truth Elements) [17] is an alive format used in both the scientific sector and real-world applications. For example, a simple table (a) is given with its PAGE XML representation (b). Each TableCell element has a defined logical position (and span) thanks to row and col attributes (rowSpan and colSpan respectively). Each visual element has its own Coords tag (d) with points creating a polygon around the object. For this thesis, we simplify all shapes to just rectangular bounding boxes (c).	24
4.3	Example from the new table dataset. In the dataset creation pipeline after detecting the table, text lines are detected and pseudo ordered. Then HTML representation is made and added to physical location information to produce PAGE XML. Table reconstruction can then be made using the cell crops. .	25

4.4	Screenshot from open-source annotation framework Label Studio web page for text line detection and structure annotation. Annotators used an external tool (table generator) to create the logical structure using HTML notation. HTML cell content corresponds to the text line IDs visible in the blue tags on top of the text line bounding boxes.	26
4.5	Example of automatic word alignment on premade textline detections. Text lines (green) were proposed by OCR layout model and approved / fixed by hand. Base lines (red) were placed automatically into 1/4 height of the text line. Word bounding boxes (black vertical lines) were aligned from OCR output logits.	27
4.6	from <i>HeritageTabNet</i> ran through <i>PubTables</i> DETR [20]	28
5.1	Class structure for OCR and TSR objects creation, loading and saving to PAGE XML. Classes and methods marked by green color are added for this thesis. The Table PAGE XML is only internal distinction that adds extra elements available, namely TableRegion and TableCell. All XML outputs from this framework should be valid PAGE XML according to the standard version 2019-07-15	30
5.2	Examples of two ground truth formulation for vertical relation class R_v . . .	33
5.3	Diagram of the proposed word relation prediction TSR architecture. $\mathbf{B}$ is batch size, $\mathbf{S}$ is sequence length of flattened image backbone output, $\mathbf{dim}$ is the given dimension of transformer vector space, and $\mathbf{N}$ is the current maximal input sequence length: number of words on the input + padding.	34
6.1	Four model sizes were trained for each experiment. The figure shows the parameter distributions in model parts. ResNet18 backbone was frozen during training. The hyper-parameter settings are detailed in table 6.2	37
6.2	Accuracies of word relation prediction models trained on <i>PubTables-1M</i> train set and measured on <i>PubTables-1M</i> val set. Predicted relation logits were thresholded with 0 threshold to create boolean predictions required by both DAR F1 and ARI. The DAR F1 (Directed Adjacency Relation) accuracy measures the model output and ground truth directly. The ARI (Adjusted Rand Index) measures the model output capability to split input elements into clusters same as are in the ground truth. Clusters for both ground truth and prediction are created with strong and weak connected component analysis (CCA). Strong CCA includes only simple cells (clusters being columns and rows). Weak CCA views the relations as undirected and thus is connecting joined cells with clusters (creating groups of columns/rows connected by the joined cell), simple visualization is in figure 2.4. Both ARI for strong and weak CCA are added here, because together they more closely resembles the use case in reconstruction.	38

6.3	Accuracies of word relation prediction models all trained on the train set from <i>PubTables-1M</i> [20]. „Fine-tuned“models were then fine-tuned on a mix of <i>HeritageTabNet</i> train set and <i>PubTables-1M</i> subset. Accuracies are measured on test sets of both datasets. ARI strong measures model capability of creating coherent clusters of simple cells. ARI weak measures the same and adds capability of correctly locating joined cells. Fine-tuning not only bosted accuracy on <i>HeritageTabNet</i> greatly, but more importantly did not damage the accuracies on <i>PubTables-1M</i>	39
6.4	Example visualization of fuzzy reconstruction from word relation prediction model to table structure. This example shows only row clustering for clarity. The algorithm goes like this: 1. Convert logits to probabilities. 2. Cluster the „same cell“class to create cells. 3. Ignore joined cells for now. 4. Cluster columns and rows. 5. Add joined cells to 0..N rows and columns. 6. Combine column and row clusters to create the table structure	41
6.5	Mean Average Precision and Mean Average Recall results original and fine-tuned word relation prediction models measured on <i>PubTables</i> [20] test set (left) and <i>HeritageTabNet</i> test set (right). Two refference results are added (<i>PubTables</i> DETR and <i>ClusterTabNet</i> [18]). The bounding boxes for precision and recall meassuring (columns and rows) get built from individual word bounding boxes using the word relation prediction model output and fuzzy reconstruction algorithm. The results show that even my best models have lower accuracy than reference models. There is a visible improvement with each fine-tuned model so there is deffinitely space for improvement. . .	42
6.6	Illustration of problematic separation of word relation predictions (left side) and its influence on the ARI (Adjusted Rand Index) clustering score and sensitive threshold results (right side). The prediction value is min-max normalized output logits. This can be done without sigmoid because they are mathematically in a fixed range due to the nature of predictio heads. Output logits are scaled normalized dot-product so they are in a fixed range.	43
B.1	Examples of table reconstruction result (left) with ground truth (right). The upper is correctly extracted, the lower has over-joined cells and therefore messed up later stages. mAP_50: 0.57, mAR: 0.67	50
B.2	Example of table reconstruction result (left) with ground truth (right). mAP_50: 0.704, mAR: 0.735	51
B.3	Example of table reconstruction result (left) with ground truth (right). mAP_50: 0.76, mAR: 0.64	52

Chapter 1

Introduction

Tables are a frequent part of documents such as chronicles, scientific papers, official reports, etc. Finding specific information or analyzing the data becomes costly in large quantities of documents. That’s where digitization comes into play. Documents can be scanned, becoming less prone to loss, but they are only slightly easier to work with or to find anything in them. Reading the text on the document and storing it in a text format is also possible now, using Optical Character Recognition systems (OCR). However, tabular data might become even less readable without the table structure information. Table Structure Recognition (TSR) tries to digitally recreate the input table structure, allowing existing table programs like MS EXCEL to read the data without manually re-typing the tables. Using a table recognition system, one can search, look for specific patterns, or analyze trends in amounts of documents previously unheard of.

While the recognition of typical English handwritten and printed tables appears to be pretty much solved and usable, given reported accuracies on large testing datasets approaching or surpassing 95 % detection accuracy [20, 18]. The focus is now shifting to more out-of-distribution data, where no big datasets are available, or the task seems much more advanced. Known state-of-the-art systems fail on dissimilarities such as writing styles, missing cell borders, or languages because they were not trained on such data. The solution is to create specialized datasets or systems not dependent on such visual features [13]. Another option is to rely on an Optical Character Recognition system (OCR) to provide valuable insight for the TSR model in the form of bounding boxes of detected words and their transcriptions. Such a TSR model can then focus on a relatively more straightforward task of looking for relations between the detected words/lines [16, 18]. This approach has been chosen for this thesis.

The main goal of this thesis is to create a custom architecture inspired by the ones proven to work and try to specialize it on a custom dataset *HeritageTabNet* of mostly Czech and German historical handwritten scans, as well as try to approach similar accuracy on the broadly used datasets like *PubTables-1M* [20] and cTDaR [4]. The architecture created for this thesis consists of known building blocks, namely an image backbone and positional embeddings of bounding boxes, a multimodal transformer decoder, and finally, similarity-based prediction heads for predicting word relations in several categories (same column, same row, same cell).

The **Table structure recognition** section provides a deep dive into the field with known datasets, evaluation metrics, and the selected influential papers for this thesis. The **Related literature** describes the building blocks upon which the TSR of today stands. The annotation format, process, and purpose of the *HeritageTabNet* dataset are described in the

section of the same name. The **Proposed method and implementation** section provides a detailed description of the proposed methodology and also insight into the codebase creation process. Lastly, the important juicy numbers are shared in the **Experiments and results** section.

This work is part of a grant project at the Faculty of Information Technology called Orbis pictus¹, which focuses on developing systems for easier search and machine understanding of large-scale historical data in Czech libraries.

¹[Orbis Pictus \(Reanimated book\)](#) – digitized library treasures for the creative industries

Chapter 2

Table structure recognition

Table Structure Recognition (TSR) is a specialized computer vision and document analysis sub-field focusing on understanding and recreating the 2-D structure and contents of tabular data in scanned printed and handwritten documents. Table Detection (TD) focuses on detecting tables in bigger documents, such as newspapers, chronicles, and other structured documents. These systems sometimes use Optical Character Recognition systems (OCR) to help provide more insight before actual analysis. More on that in section 2.5.

A TSR system’s result is typically a table structure in a machine-readable format. It should be straightforward to read how many rows and columns the table has, where its bounding boxes are, what structure the cells have, and if any joined cells exist. The typical output of such a system consists of a table representation in HTML or XML notation or bounding boxes for each row and column.

Table extraction pipeline. More steps are needed for a real-world table extraction system (TE). Tables must be detected using table detection (TD); logical structure must be inferred using some TSR block. Depending on the TSR model, an OCR transcription has to be mapped to the logical structure, creating a table reconstruction. Some TE systems perform functional analysis (FA) of the table, classifying parts of the table into column headers and projected row headers for a better understanding of the data.

Use cases for TE systems. **Document Digitization in Archives** is the most straightforward use case. It can help preserve historical documents, academic papers, and business records. TSR facilitates digital preservation without losing structural data and the searchability of archival materials. **Healthcare Records Management** in medical institutions extracts and organizes patient data from medical reports, which helps with electronic health record management and enables better patient data analysis and treatment planning. Organizations can leverage TSR for **Business Intelligence**: processing invoices, purchase orders, and inventory reports. It can help extract product details, pricing, and other business data, streamlining decision-making processes.

2.1 Datasets

Four existing datasets are mentioned here. Large generated datasets *PubTables-1M* [20] and *PubTabNet* [26], and smaller scanned datasets *GloSAT* [27] and *cTDaR* [4]. A summary of numbers and ground truth formats is available in table 2.1, and input examples are in fig-

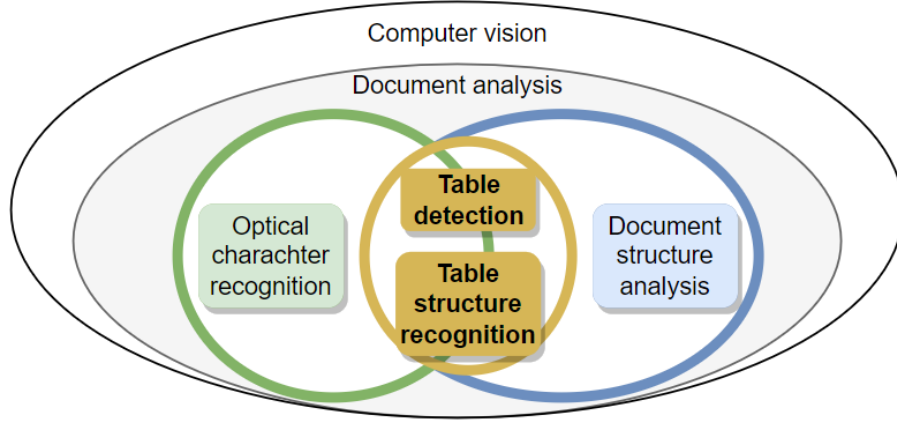


Figure 2.1: Table Detection (TD) and Table Structure Recognition (TSR) is a sub-field of Document structure analysis, sometimes using Optical Character Recognition systems. The main goal is to detect, understand, and recreate tabular data in scanned documents.

Table 2.1: Summary of TSR datasets. Large generated datasets *PubTables-1M* [20] and *PubTabNet* [26], and smaller scanned datasets *GloSAT* [27] and *cTDaR* [4]. Lastly, the new dataset introduced for this thesis *HeritageTabNet* described in section 4.

	# Tables	GT format	Image origin
PubTables-1M	948 K	Bounding boxes*	Generated from PDF
PubTabNet	510 K	PDF	Generated from PDF
cTDaR trackB	900	ICDAR XML	Handwritten & Scanned
GloSAT	500	PAGE XML	Handwritten & Scanned
HeritageTabNet	630	PAGE XML	Handwritten & Scanned

*Bounding boxes in different categories: words, tables, rows, columns, spanning cells, column header, projected row header

ure 2.2. In addition to the datasets described in this section, a new dataset *HeritageTabNet* created for this thesis is described in section 4.

PubTabNet. PubTabNet¹ described in [26] originates from the PMCOA PDF data corpus. Images are rendered PDFs, and ground truth is in PDF files. With over 500K tables, it was the largest available dataset for a few years. Although the size is incomprehensible, the tables are only generated, so usability for a real use case is in question.

PubTables-1M. *PubTables* [20] in 2022 introduced a brand new large dataset with tables in PDF and VOC format, including the source PDF, sub-table element locations, and word locations and transcriptions. All the information needed for table structure recognition (with row, column, and spanning cell classes) and functional analysis (with column header and projected row header classes). With nearly a million tables, it almost doubled the size of the previous biggest table dataset *PubTabNet*.

The tables are taken from the PMCOA PDF data corpus, so the dataset consists of rendered tables. Again, it's only generated tables, not handwritten, nor images distorted

¹github.com/PubTabNet

Variable	Hazard ratio	95 % CI	p value*
Age (median)			0.716
≤69	1.000		
>69	0.839	0.310-2.268	
Gender			0.142
Male	1.000		
Female	0.426	0.152-1.190	
Type of surgery			0.010
Low anterior resection	1.000		
Abdominoperineal resection	3.140	0.919-10.725	
Tumor location			0.710
Upper rectum	1.000		
Middle rectum	1.267	0.381-4.213	
Low rectum	1.716	0.419-7.026	
Grade of differentiation			0.936
G1	1.000		
G2	1.933	0.416-3.423	
G3	1.119	0.137-9.137	
Histologic type			0.299
Adenocarcinoma	1.000		
Adenocarcinoma with mucinous features	0.381	0.086-1.514	
Depth of tumor invasion			0.925
T3	1.000		
T4a	0.919	0.316-2.673	
T4b	0.745	0.172-3.223	
Tumor size			0.529
≤4 cm	1.000		
>4 cm	0.594	0.214-1.651	

PubTabNet

MONTH	Temperature in Shade		MEAN OF						TOTAL OF		REMARKS
	Highest Reading	Lowest Reading	Thermometer		Thermometer in Shade		Days in Shade	Number of Wet Days			
			Day Bulb	Wet Bulb	Day Bulb	Wet Bulb					
January	30.126	82.0	25.5	41.4	39.7	37.6	44.5	38.0	1.8	1.3	
February	30.076	85.5	25.0	41.3	39.4	37.1	48.2	83.7	4.8	.3	
March	29.461	88.0	24.8	42.6	41.3	39.7	47.1	86.2	6.0	.22	
April	29.933	87.0	32.5	47.4	46.2	87.4	41.8	1.9	1.1	.15	
May	30.046	74.8	38.0	66.8	63.4	58.8	52.9	48.3	1.8	.6	
June	29.947	72.0	44.0	88.9	85.3	82.1	64.1	44.7	2.7	.16	
July	29.974	78.5	49.0	81.4	80.0	87.1	64.6	84.8	1.8	.9	
August	30.016	83.0	48.0	66.4	62.3	60.0	71.8	84.0	2.3	.13	
September	30.014	85.0	42.0	88.3	84.3	80.8	63.2	88.0	1.2	.14	
October	29.776	88.0	28.0	84.3	82.1	80.0	88.7	47.3	4.6	.20	
November	30.004	89.0	26.0	43.8	41.8	39.7	45.3	36.9	0.4	.7	
December	29.461	85.5	23.0	42.1	40.8	39.2	48.7	88.5	5.2	.15	

GloSAT

Birthweight distributions and odds ratios for breast cancer in African-American and white women combined								
	Minimally adjusted OR ^a				Fully adjusted OR ^b			
	Cases	Controls	OR ^a	95% CI	Cases	Controls	OR ^b	95% CI
Full birthweight dataset	n = 196	n = 167			n = 191	n = 161		
Lower tertile ^c	73	57	0.9	0.6-1.6	72	55	1.0	0.6-1.7
Central tertile	70	54	Ref.		69	51	Ref.	
Upper tertile	53	56	0.7	0.4-1.2	50	55	0.7	0.4-1.2
Mean ± SD (g)	3282 ± 550							
Median (g)	3232							
Range (g)	1021-4621							
Restricted dataset ^d	n = 147	n = 100			n = 143	n = 97		
Lower tertile ^e	59	43	0.9	0.5-1.7	58	41	1.0	0.5-0.9
Central tertile	48	32	Ref.		47	31	Ref.	
Upper tertile	40	25	1.0	0.5-2.1	38	25	0.9	0.4-1.9
Mean ± SD (g)	3210 ± 482							
Median (g)	3232							
Range (g)	2041-4621							

*Adjusted for age, race and sampling fractions. ^bAdjusted for age, race, sampling fractions, history of previous biopsy, maternal age, and adult body mass index >25 kg/m². ^cTertiles are race specific with subjects derived from controls. White women: <3000 g, 3000-3400 g, >3400 g. African American women: <3146 g, 3146-3486 g, >3486 g. ^dBirthweight measured in pounds and ounces and participant delivered in a medical

PubTables-1M

Reg. No.	Date of joining	Name	Address
✓ 12.25.12.5.21	✓	Mohamed, Babak	✓ Hedonore, ✓ Limerick
✓ 12.26.12.5.21	✓	Kiogh, Mamas	✓
✓ 12.27.12.5.21	✓	Ryan, John	✓ Knochatona
✓ 12.28.12.5.21	✓	Bennett, Thomas	✓ Chacacatunk
✓ 12.29.12.5.21	✓	Wash, Patrick	✓ Quisque, Bonifacio, (Bar)
✓ 12.30.12.5.21	✓	Alvares, Corina	✓ Traction, Chacacatunk, Limerick
✓ 12.31.12.5.21	✓	Cough, Richard P.	✓ Coston, Map
✓ 12.32.12.5.21	✓	Kief, David	✓ Aldangas, Montanowen
✓ 12.33.12.5.21	✓	Cough, Michael P.	✓ Knoch, (Bar)
✓ 12.34.12.5.21	✓	Barth, Patrick	✓ Barth, Andrew, (Bar) Woodford
✓ 12.35.12.5.21	✓	Wish, Thomas	✓ Slick, Glasanradap, (Bar)
✓ 12.36.12.5.21	✓	Wish, Peter	✓ Limerick, (Bar) Glasanradap
✓ 12.37.12.5.21	✓	Wish, Patrick	✓ Wishing, (Bar) Glasanradap
✓ 12.38.12.5.21	✓	Wish, John	✓ Wishing, (Bar) Glasanradap
✓ 12.39.12.5.21	✓	Wish, Patrick	✓ Wishing, (Bar) Glasanradap
✓ 12.40.12.5.21	✓	Wish, Patrick	✓ Wishing, (Bar) Glasanradap
✓ 12.41.12.5.21	✓	Wish, Patrick	✓ Wishing, (Bar) Glasanradap

cTDaR TrackB

Figure 2.2: Examples of dataset described in this thesis: Large generated datasets *PubTables-1M* [20] and *PubTabNet* [26], and smaller scanned datasets *GloSAT* [27] and *cTDaR* [4].

by cameras. This paper comes with a new thought of making table annotations more informative by defining the canonical structure of tables, meaning empty cells get detected as joined, and row headers for multiple rows also get connected to one joined cell. This principle is not further discussed in this thesis because the proposed architecture works only with non-empty cells by definition. Empty cells are only a byproduct of reconstruction, so they do not affect accuracy.

The ground truth for the object detection task comes in the form of VOC (Visual Object Classes) XML annotations for TSR (columns, rows, and spanning cells), functional analysis (column headers and projected row headers), and word bounding boxes and transcriptions. The actual table structure is constructed with a straightforward algorithm called intersection cell creation and word assignment described more in section 5.4

GloSAT. This dataset consists of 500 historical tables from meteorological measurement logbooks. The tables are provided in two XML formats, ICDAR XML compatible with cTDaR dataset, and so-called Transkribus format, which is PAGE XML. XML formats provide both logical structure and cell locations. It is worth mentioning that cell bounding boxes are made by outlining the border lines, which is different from both *HeritageTabNet*

and *cTDaR*, which provide cell bounding boxes around the text lines, not the whole cells. Henceforth, object detection metrics are not directly comparable.

cTDaR. The ICDAR 2019 competition on the TSR dataset consists of 900 historical (mostly handwritten) and modern (printed) tables in ICDAR format. All the tables are scanned.

2.2 Evaluation metrics

The evaluation metrics vary depending on the task formulation. For this thesis, the following have been chosen as they align with the chosen task. Directed adjacency relations (DAR) [6] F1 measure will be used for validation during training since it can compare the model output directly with the ground truth. Average precision + recall of sub-table elements (rows and columns) is an object detection metric and will be used to compare results with known results in previous work. Tree edit distance, first described in [26], evaluates the reconstructed table structure with the content at once, making it the ideal metric for measuring both OCR and TSR parts of the given system in an end-to-end manner.

Directed adjacency relations (DAR) [6]. This metric can compare the TSR model output directly, so no post-processing algorithms influence it. For an input of N detected words, an output $N \times N$ matrix is generated, representing the directional relation between cells for each relation class. In this thesis it is primarily same cell, same column, and same row relation. If the model output is probabilistic it needs to be thresholded to **True** or **False**. Output adjacency relation at $[X, Y]$ is then correct if it equals ground truth adjacency relation at $[X, Y]$. Most of the values in matrices are negative (most of the cells are not connected), so only positive relations are taken into account to measure precision, recall, and F1 score using the following formulas:

$$\begin{aligned} \text{Precision} &= \frac{\text{correct relations}}{\text{ground truth relations}} \\ \text{Recall} &= \frac{\text{correct relations}}{\text{detected relations}} \\ \text{F1} &= 2 \times \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}} \end{aligned} \tag{2.1}$$

Adjusted Rand Index (ARI). The Adjusted Rand Index² measures the correctness of clusters against the ground truth. It requires a disjunctive cluster label for each element. The basic Rand Index (RI) is calculated as seen in 2.2, where TP is the number of true positive cluster assignments, TN is the number of true negatives, FP is the number of false positives, and FN is the number of false negatives. The RI has the $< 0, 1 >$ range, where 0 means the worst and 1 means correct cluster assignment. The Adjusted Rand Index (ARI) then provides random assignment analysis and scales the RI to the $< -0.5, 1 >$, where 0 corresponds to random assignment, 1 means correct, and less than 0 means worse than the random assignment.

²scikit-learn.org/sklearn.metrics.adjusted_rand_score

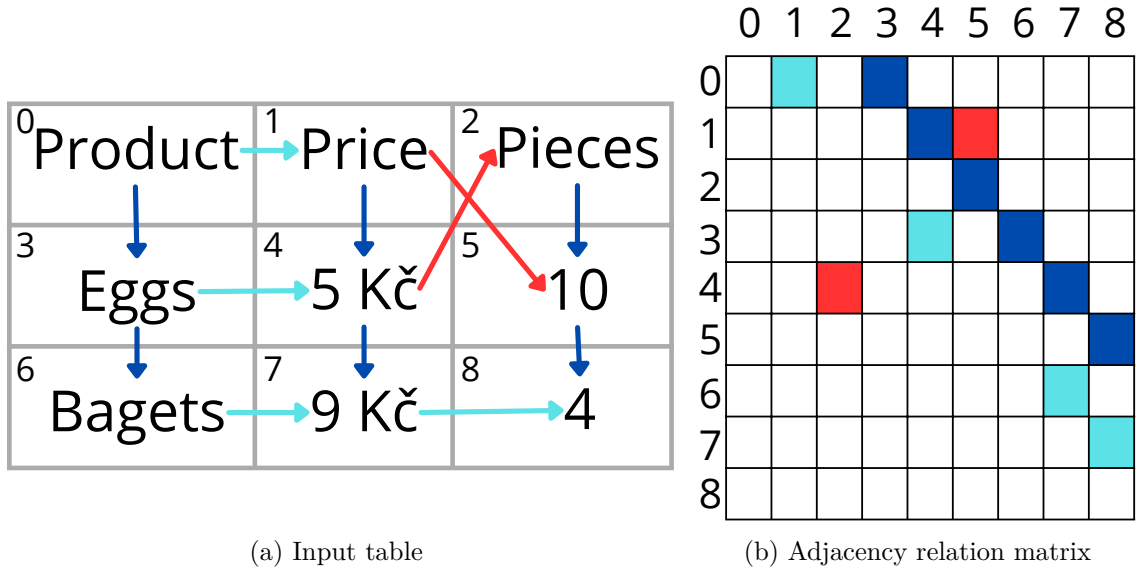


Figure 2.3: Example of adjacency relations between cells to measure F1 DAR (Directed adjacency relations) [6]. For N cells, a matrix N x N is generated, representing the directed nearest neighbor relation between cells (vertical and horizontal). For example, the cells 0 and 1 are horizontal neighbors, so there is True in the horizontal neighborhood matrix on the position [0,1]. In total, this table has 12 relations, 12 detected relations, 10 correctly detected relations (dark blue and light blue), and 2 incorrectly detected relations (red).

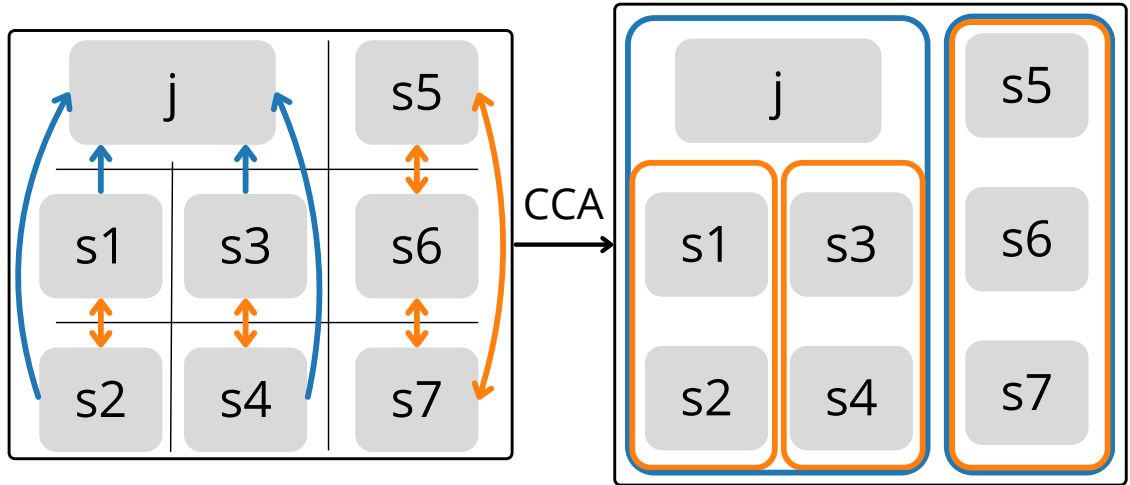


Figure 2.4: Example visualization of **strong** and **weak** connected components in a table structure with clustering neighbours relations between cells.

$$RI = \frac{TP + TN}{TP + FP + FN + TN} \quad (2.2)$$

$$ARI = \frac{RI - Expected_RI}{max(RI) - Expected_RI} \quad (2.3)$$

In this thesis, a connected component analysis will be used to create strong³ and weak⁴ connected components (cells, columns, and rows) from the adjacency matrix, which should, for the given task, provide semantically more accurate evaluation than the F1 measure used for raw model output evaluation. An example of strong and weak components is in figure 2.4

Tree edit distance similarity (TEDS). TEDS is first introduced in [26] and compares both the logical structure of the table as well as its text content. The input is an HTML-like representation understood as a tree, and the algorithm counts the edit distance as the number of insertions, deletions, and substitutions of nodes in the tree. Each operation has a cost of 1, except when the text content of the nodes differs. In that case, the cost is normalized Levenshtein similarity (0 to 1). TEDS between two trees is computed like this, where EditDist is tree-edit distance and $|T_i|$ is the number of nodes in T_i

$$\text{TEDS}(T_a, T_b) = 1 - \frac{\text{EditDist}(T_a, T_b)}{\max(|T_a|, |T_b|)} \quad (2.4)$$

The code to measure TEDS and an in-depth description were released together with the PubTabNet dataset⁵ in [26]. It is also used as one of the main metrics in *SPRINT* paper [13] described in section 2.4

Average precision, average recall. A few object detection metrics also have to be mentioned. Ground truth and output bounding boxes are coupled with IoU thresholds (intersection over union of their area), and then known metrics precision and recall are used (similar to the description in equation 2.1. Specifically for TSR, the object detection approaches detect sub-table elements (rows and columns), and use object detection metrics as a direct indicator of accuracy (as seen in *PubTables* [20]). Systems with OCR as an input do not directly predict the bounding boxes. But, to be able to compare the results between different approaches, sub-table element bounding boxes are created as a region surrounding all cells in a given sub-table element. The bounding boxes are created by getting the minimum and maximum coordinates of all words inside (as seen in *ClusterTabNet* [18]) with the exception of joined cells, which get counted only in X coordinates for columns (and Y for rows) to prevent stretching multiple boxes to the same one. An example of this is available in figure 2.5

2.3 PubTables (2022)

PubTables [20] in 2022 introduced a brand new large dataset with nearly a million tables in PDF format. More about the dataset is in chapter 2.1. This paper approaches the TSR problem in an object detection way while attending to three subtasks. Table detection (TD) is a straightforward concept of detecting a table on a page with a bounding box. In their formulation, table structure recognition (TSR) aims to detect bounding boxes of sub-table structures: rows and columns. The same applies to functional analysis (FA), which aims

³In a strongly connected component, all nodes have a path between each other, edges are treated as directed

⁴In a weakly connected component, all nodes have a path between each other, but edges are treated as undirected

⁵github.com/PubTabNet/

Shopping list for today and tommorow		
Product	Price / piece	How many
Eggs	5 Kč	20
Bagets	9 Kč	
Sausages	15 Kč	10

Figure 2.5: Example visualization of column and row bounding box creation for applying object detection metrics to word relation prediction tasks. The bounding boxes are created by getting the minimum and maximum coordinates of all words inside (as seen in *ClusterTabNet* [18]) with the exception of joined cells, which get counted only in X coordinates for columns (and Y for rows) to prevent stretching multiple boxes to the same one.

to detect bounding boxes of sub-table structures based on their semantic and topological meaning. One model detects tables on a page using two classes: `table` and `table rotated` (for tables rotated counterclockwise 90 degrees). The second model takes the table crop, analyzes both TSR and FA at once, and detects objects in 5 classes: `column` and `row` for TSR, `column header`, `projected row header`, `spanning cell` for FA. An example is in figure 2.6. Two model architectures are compared: Detection Transformer (DETR) [2] and Faster-RCNN [19] with varied results.

Firstly, direct results without post-processing are tested with object detection metrics average precision (AP) and average recall (AR) based on intersection over Union (IoU) thresholds, which are described in section 2.2. DETR outperforms Faster R-CNN for all three subtasks, demonstrating superior performance for table localization and understanding.

Secondly, the TSR performance of reconstructed tables is tested using two TSR-specific metrics. DAR metric [6] compares directed adjacency relations with F-score. GriTS metric [21] estimates partial table correctness in three categories (cell topology and location, and content). Both metrics are described more in section 2.2. DETR outperforms Faster-RCNN on these metrics too, and is thus deemed to be superior for TSR systems.

Reconstruction with intersection cell creation and word assignment. After detecting sub-table elements, a reconstruction algorithm is needed. In this paper, detected objects are first dilated to cover the maximal non-overlapping area of the table (each pair of adjacent rows resp. columns expands their boundaries until they meet halfway, filling the empty space). Secondly, intersections of rows and columns are converted to cell bounding boxes, creating the basic table structure of simple cells. Spanning cells are then added via intersection to finish the predicted table structure. Individual cells are classified into column headers, projected row headers, or default cells using intersections with FA classes. Optional detected words with external OCR-like tools get assigned either to joined cells or simple cells via the maximum intersection with cell relative to the word bounding box size.

Group	ASD ($n = 11$)			TD ($n = 8$)			Mann-Whitney test	
	Q ₁	Median	Q ₃	Q ₁	Median	Q ₃	U	p
Android robot								
Feel enjoyable	4	5	8	2	3.5	4.75	16.50	0.02*
Feel embarrassed	3	5	7	2.5	5	7.5	42.00	0.90
Feel stressed	1	3	5	2.25	3.5	6	36.50	0.55
Feel bored	1	2	3	2	3	5.75	27.00	0.18
Visually simple robot								
Feel enjoyable	5	6	9	2	5	5.75	17.50	0.03*
Feel embarrassed	1	4	6	1	3	3	31.50	0.31
Feel stressed	1	3	5	1	2.5	3.75	39.00	0.72
Feel bored	1	2	4	1.25	3	3.75	35.00	0.18

Row (odd)

Row (even)

Column (odd)

Column (even)

Spanning cell

Column header

Projected row header

Figure 2.6: This image is taken from *PubTables* [20], where all three table extraction sub-tasks are handled with an object detection approach (table detection, structure recognition, and functional analysis). For TSR and FA, the proposed detection model detects objects in 5 classes at once: column and row for TSR, column header, projected row header, and spanning cell for FA. Two model architectures are compared in the paper: Detection Transformer (DETR) [2] and Faster R-CNN [19] with varied results discussed in chapter 2.3.

2.4 SPRINT (2024)

Script-agnostic Structure Recognition in Tables (SPRINT) [13] from 2024 focuses on logical structure recognition (cell shape and merging). It does not aim to create tables or cells’ physical structure (bounding boxes) as it aims to outsource this task with an already existing detection-based model and correct it. The proposed Img2Seq architecture creates a logical structure represented as an HTML-like text sequence called Optimised table-structure language (OTSL) originating in [15]. The accuracy is then measured using the tree edit distance (TED) variant for trees on *PubTables-1M*, *PubTabNet*, and *FinTabNet* datasets described more in section 2.1. This metric is described in section 2.2.

It was proven to match the accuracies of state-of-the-art systems and surpass them in out-of-distribution data, such as the new MUSTARD dataset consisting of tables in English, Chinese, and 11 Indian languages. From an architectural point of view, this paper doesn’t bring much innovation. Images are downsampled and resized to a square. An encoder-decoder transformer is then used to produce the output sequence of tokens/letters. The innovation comes in proving insufficient accuracies of SOTA systems on non-English data, providing unseen data to measure that, and testing known architecture in a new task definition to create a model hopefully prone to the mentioned problems. The proposed architecture pipeline can be seen in figure 2.7.

This paper claims that known object detection-based TSR models need large training datasets and do not handle changes in language or writing styles well. To prove this claim,

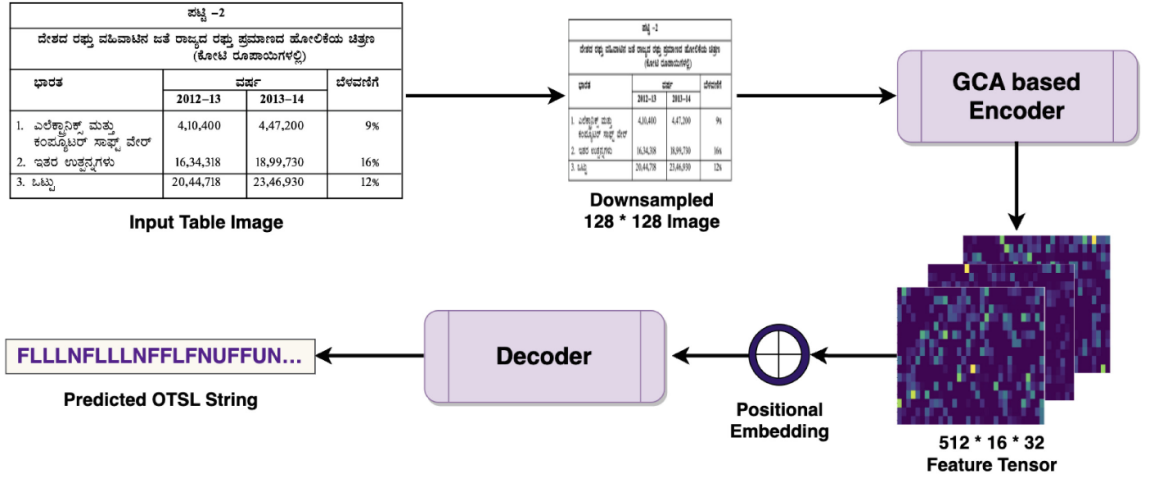


Figure 2.7: This image is taken from SPRINT [13], where table logical structure recognition is attended with an Img2Seq model. The table input is downsampled and resized to a square. An encoder-decoder transformer is then used to produce the output sequence of tokens/letters. Output vocabulary is made out of 5 tokens/letters (F, L, U, X, N). F (regular cell), L (cell merged with left neighbor), U (cell merged with upper neighbor), X (cell merged with both upper and left neighbor), N (new line/row of the table). GCA means global context attention encoder described in [14].

a new dataset, MUSTARD, was created for testing and fine-tuning purposes. The dataset includes 1 428 tables with Latin and non-Latin alphabets in 13 languages total: English, Chinese, and 11 Indian languages.

Takeaways. The main takeaway is the metric for comparing HTML output with table cell structure and contents. The paper also introduces an interesting combination of the Img2Seq model and already used detection models like *PubTables* DETR [20] to boost accuracy further.

2.5 ClusterTabNet (2024)

The ClusterTabNet paper [18] takes the task from a clustering perspective. The proposed architecture utilizes word bounding boxes from OCR output and predicts relations between pairs of words in several classes (belonging to the same row, column, header, and table) using a transformer encoder model. Resulting relations are clustered to create a directed relation-based table structure, which is then connected with the word transcription to provide table reconstruction. The paper demonstrates performance on multiple known datasets and achieves similar or better accuracy while requiring a significantly smaller model (5M parameters versus 29M parameters of DETR). The results are compared using TSR metric F1-DAR and object detection metrics (described in section 2.2) on *PubTables-1M*, *PubTabNet*, and *FinTabNet* datasets described more in section 2.1 The architecture proposed in this paper serves as a heavy inspiration for this thesis and is described more in figure 2.8.

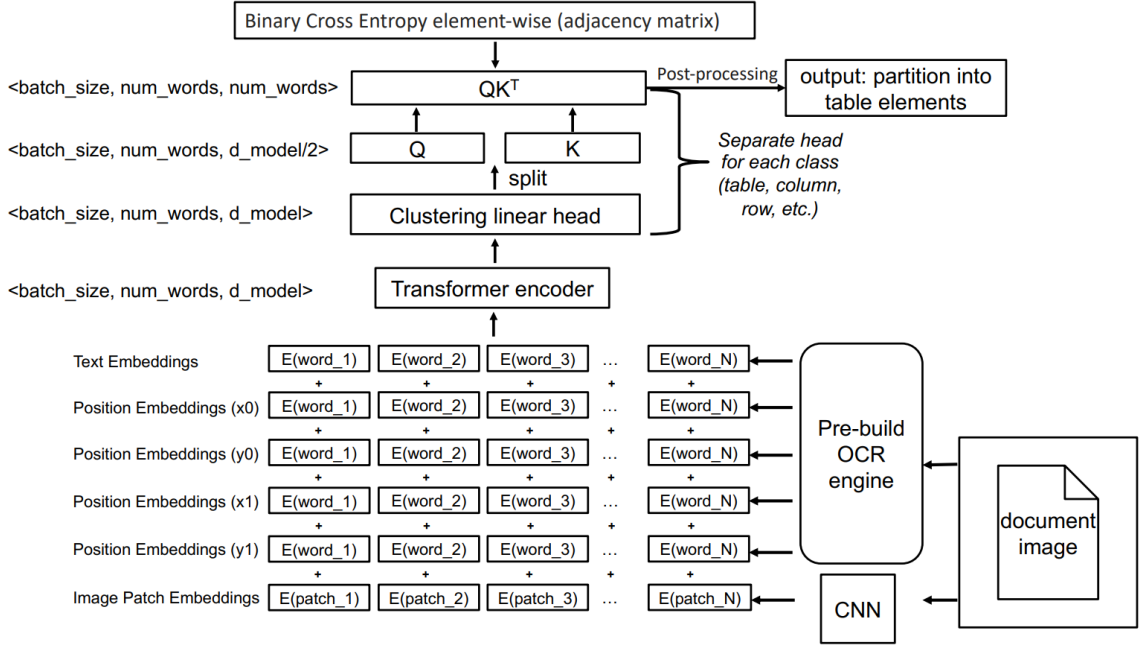


Figure 2.8: This image is taken from ClusterTabNet [18], where table detection and table structure recognition are attended. The architecture relies on a pre-build OCR engine to detect and transcribe words. Text embedding is then made for each word preprocessed with a simple replacement algorithm („The date is 12.9.“ -> „Aaaaaaaa11,1,“) and position embeddings for each coordinate (x0, y0, x1, and y1). A CNN backbone preprocesses the input image cut into crops around words (with padding). A transformer encoder processes all this information, and the last step consists of several parallel similarity-based prediction heads for each class of relation between cells. The result of this architecture is a similarity score for each word pair in every class. For an input of N words and C prediction head/relation classes (same row, column, table), the output is $C \times N \times N$.

Deep dive into evaluation. The object detection metrics used in this paper are slightly modified. The modification comes with working with OCR detection as input. Whereas typical object detection models like *PubTables* DETR [20] directly detect sub-table elements (rows, columns...) and then dilate them to fill the space between them, this architecture only clusters detected words into sub-table elements without dilating or including the space around the words' bounding boxes. These concrete number results are, therefore, not directly comparable. The authors also trained their own models based on referenced papers and code bases provided with similar training in an effort to compare results directly. Namely, *PubTables* DETR model on a mix of datasets. Similar accuracies as provided in the reference papers were achieved, so the difference is proven to be negligible.

Take aways. This paper introduces a new architecture capable of table detection and structure recognition of multiple tables on a page at once. It utilizes the power of an OCR word extractor to „do the heavy lifting“ of detecting and transcribing words. Using this principle, smaller models are capable of achieving accuracies similar to those of their bigger counterparts working without an OCR preprocessor.

Usage in a real system. The usage of this approach is relatively straightforward. Table reconstruction is created by first thresholding word pair relations and removing detected spanning cells (by majority unidirectional relations). Clusters (columns and rows) are then searched as connected components. The spanning cells are then added back to finish the logical structure. The output can be converted to a table format, such as HTML or XML, typical for OCR systems, such as PAGE XML described in section 4.1.

2.6 Discussion

There are many ways to try to estimate table structure, ranging from graphical element analysis and other hard-coded methods, through machine learning methods to deep learning methods explored in this work. The deep learning methods can further approach the task differently. Some papers approach it as an object detection task, looking for sub-table structures (rows, columns, headers) [20]. Others focus on retrieving structure directly using HTML-like simplified encoding [13] using Img2seq models. Many of these have only the image as an input, without any additional information, and thus have to be trained on large datasets to understand the picture with all visual features. They do not handle changes against the training dataset well, such as different text fonts, language, or table style, as discussed in SPRINT [13]. Furthermore, Img2seq and object detection methods rely heavily on post-processing algorithms or even other TSR models, which can bring new errors that hinder the real accuracy of the TSR model itself.

The most significant advancements in recent years (in both accuracy and usability) are visible with the use of preliminary OCR analysis as a way to extract text information and detect words/lines/cells, delegating this part away from the actual TSR model. Models using OCR information as an input, as well as the original image, tend to need less training data and fewer parameters to achieve similar results as models relying only on the image as input [18]. This also introduces modality, as one can choose between available OCR models for their specific application without needing to create more table-specific datasets. Having the OCR already baked in helps to define a table with all its content, not just structure.

TSR systems relying on OCR transcription and bounding boxes as input can also have different goals. The first option is detecting physical structure using a clustering method [18]. The second option is to predict adjacency between cells directly using relations between detected lines/words in several categories (words are in the same cell or are the closest vertical or horizontal neighbors) [16].

The main goal of this thesis is to hop on the trend of using OCR information as an input and predicting the physical structure of the table using cell adjacency relations to define table structure, reconstruct the table into 2D shape and, together with the OCR transcription, create an XML table representation.

Chapter 3

Related literature

Table structure detection (TSR), as known today, builds upon achievements of the past decades and new inventions in the field. Convolutional Neural Networks are used for image processing, transformer architectures with attention mechanisms ensure global context, and some approaches use multimodal transformers, combining image input with other modalities, such as text sequences or geometry. These concepts are widely known in the machine learning community, so this chapter is a summary focusing on topics integral to the thesis.

The architecture used in this thesis consists of a CNN backbone, transformer decoder, and word relation prediction heads. The architecture is described more thoroughly in chapter 5.3.

Convolutional Neural Networks as a backbone. Convolutional Neural Networks (CNNs) have a unique way of understanding 2D relations between pixels and pixel blocks, thanks to their unique square convolutional kernels (3x3, 5x5, ...). CNN architectures like ResNet [7] with mostly 3x3 kernels are used in many applications as visual feature extractors. Resulting visual features represent parts of images in a latent vector space, making it an ideal input for the rest of the architecture.

The latest trend in TSR is to use some of the ResNet variants as a backbone to extract visual features. While most architectures process the whole image with a backbone and add positional embeddings to introduce locality [20], some have suggested using only the cell/text line crops (with some padding) detected by a preprocessor as a way to make the model more focused on cells/text lines and recognizing cell borders and spaces between cells [18]. The first option is explored in this thesis. More about the ResNet architecture is available in *Deep Residual Learning for Image Recognition* [7]

Transformers. A transformer introduced in *Attention is all you need* [22] is one of the sequence-to-sequence (seq2seq) architectures capable of auto-regressive sequence generation (meaning a new token is generated while having the context of all previous sequence tokens). It is used most notably for NLP tasks, such as language translation and summarization, and in widely known applications like ChatGPT or DeepSeek. With a unique technique called attention, it can remember large numbers of previous tokens as context. In a typical seq2seq task, the input consists of a sequence of words/tokens or generally some features encoded to the transformer’s vector space. The output consists of output logits with scores for every possible output sequence element.

Image-to-sequence architectures (img2seq) take image parts (patches) as an input, allowing the transformer to understand the image features, which can be used in various

tasks such as OCR [10]. In recent work, images are typically first preprocessed with a CNN backbone. [23, 2, 24]

3.1 Detection transformer (2020)

Detection transformer, introduced in *End-to-end object detection with transformers* [2] is a relatively new method of object detection understood as a direct set prediction problem using a transformer encoder-decoder architecture [22]. This architecture makes several hand-designed components obsolete like spatial anchors and non-maximal suppression. One of the main advantages is that, at its core, it uses widely used ResNet [7] and transformer blocks, and parallel capabilities of prediction. The architecture is evaluated using object detection metrics and the COCO dataset where it outperforms the previous state-of-the-art architecture Faster-RCNN on big objects, but falls behind on small objects. Two model variants are introduced with different backbones. DETR (with ResNet50 and 41M parameters total) and DETR-R101 (with ResNet101 and 60M parameters total) where the bigger variant achieves better results at the cost of computational cost.

The DETR architecture is demonstrated on a general-purpose object detection dataset and is expected to work on specialized detection tasks as well. This is proven in the *PubTables* paper [20] described in section 2.3.

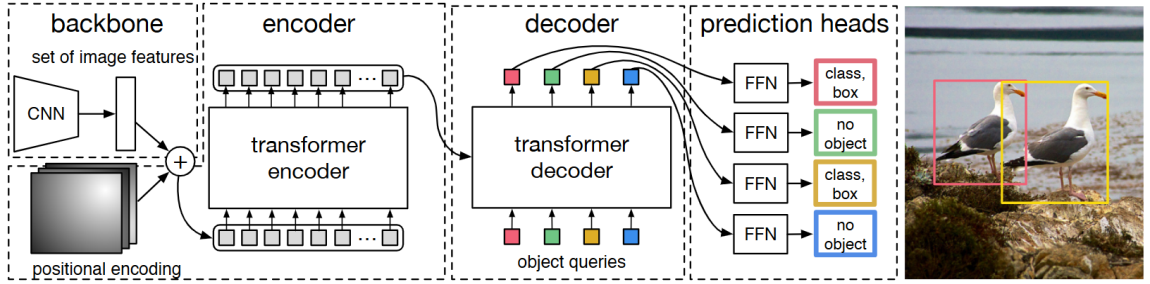


Figure 3.1: This image is taken from *End-to-end object detection with transformers* [2], which introduces a detection transformer architecture (DETR) for object detection in the form of direct set prediction. Image is processed with ResNet backbone, and positional embedding is added. Latent embeddings are then created using a transformer encoder. A transformer decoder then takes object queries (a small fixed number of learned positional embeddings). Output comes through a feed-forward network (FFN) to produce predictions (class and bounding box) or a „no object“ class.

3.2 DLAFormer (2024)

The *DLAFormer: An End-to-End Transformer For Document Layout Analysis* paper [24] introduces a novel approach to document layout analysis. While not directly related to TSR, the architecture investigated in this thesis is heavily inspired by the DLAFormer. The paper introduces a new formulation of a few document layout analysis sub-tasks. Namely, text region detection, logical role classification, and reading order prediction. The introduced architecture can address all of these formulated as relation prediction problems. Other approaches often handle these sub-tasks separately in multi-branch or multi-stage scenarios,

requiring specialized architecture. Experiment results show superiority on widely used benchmarks, DocLayNet and Comp-HRDoc.

At its core, the DLAFormer implements an encoder-decoder transformer inspired by DETR [2] (described more in section 3.1). Input images are preprocessed by a CNN backbone and separately with an OCR engine. The resulting matrix defines relations between input text lines and detected graphical objects. These relations are of two categories: intra-region relations connecting text lines in one region and inter-region relations defining reading order on a region level. Each graphical element is also classified into one of several DLA classes (heading, paragraph, caption, table, footnote, ...). An example is provided in figure 3.2

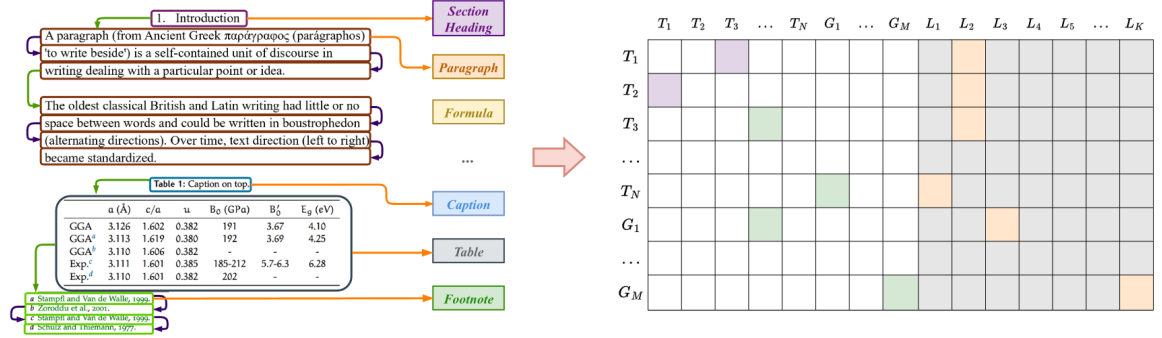


Figure 3.2: This image is taken from *DLAFormer: An End-to-End Transformer For Document Layout Analysis* paper [24], which introduces a novel approach to document layout analysis (DLA) and its several subtasks. Clustering text lines into regions and reading order prediction is visible given purple lines (defining a reading order of lines) and green lines (defining a reading order of regions, as well as text lines not belonging to the same region). All this using exclusive relations between graphical elements visible on the right side. T_i are text lines detected by an OCR preprocessor, and G_i are graphical objects proposed by the encoder part without OCR knowledge. All these graphical elements are also classified into logical role classes.

The relations in this paper are defined as exclusive and directed. Each graphical element has to have one relation, and the last one has a self-loop. This works perfectly for 1-D characteristics of reading order. This approach is upgraded in this thesis to non-exclusive relations to provide a table structure, which is 2-D by definition.

Takeaways. This thesis is made possible by the research of this paper for its unique and innovative usage of multimodal transformers with an OCR preprocessing and relation formulation of the tasks in previously unheard ways.

Chapter 4

HeritageTabNet dataset

The dataset review in chapter 2.1 discusses existing datasets of both generated and scanned table images. The goal of this thesis is to implement a system capable of recognizing historical Czech tables. To test and possibly fine-tune different models and the new architecture introduced in this thesis, a new dataset has been made consisting of very specific domain samples: Czech and German historical archives and libraries. This thesis introduces a new multi-purpose table dataset *HeritageTabNet*, which consists of 342 scanned pages with 633 tables annotated in a well-defined and widely used PAGE XML format¹. The annotations consist of bounding boxes for the tables, cells, and text lines with OCR transcriptions. Secondly, it defines the logical cell structure and classifies cells into three classes. The PAGE XML format was chosen as it allows natively saving all the information needed. The simplified structure of the PAGE XML table can be seen in figure 4.2. Examples of the tables are available in figure 4.1.

The dataset consists of 67 difficult tables (having one or more spanning cells). The word count ranges from 4 to 378, and the cell count ranges from 4 to 300. All tables have 2-21 columns and 2-67 rows. The biggest col span is 8 and the biggest row span is 4.

The dataset was annotated in a manual correction of the automatic proposal framework, where the annotators analyzed and fixed proposals created by various models and algorithms for each stage of the process. Creating the dataset was made possible due to finances kindly provided by the Orbis Pictus project². Three student annotators were acquired and paid by the hour, thanks to the financing. Managing different steps of the dataset creation pipeline and ensuring consistency between annotators proved to be a good experience in communication and management skills.

The scanned pages come from historical handwritten chronicles, registry books, and other official reports, where tables play a critical role in capturing the structural essence of the data and relations between individual cells. Pages themselves come from a mix of openly available data from Czech libraries, such as KNAV³, MZK⁴ and NKP⁵. These libraries have parts of their data openly available in web applications kramerus⁶ and digiknihovna⁷

¹github.com/PAGE-XML

²Orbis Pictus (Reanimated book) – digitized library treasures for the creative industries

³Knihovna Akademie věd ČR

⁴Moravská zemská knihovna

⁵Národní knihovna České Republiky

⁶kramerus.nkp.cz

⁷digiknihovna.cz

Průběh	Počet zářků	Prospěch					Poznámka
		1	2	3	4	5	
I.	35	12	12	7	-	4	
II.	47	15	13	9	4	6	
III.	23	10	9	5	3	-	4 žáci nestihli - nemu.
IV.	23	4	6	11	1	4	2 žáci nestihli - nemu.
celkem	133	41	40	32	8	14	

Předpis:		Dodáno:	
Cukrovka	22 692 g	23 770 g	
Čekanka	2600 g	3 250 g	
Brambor	1044 g	1547 g	
Maso hovězí		392 g	
Maso vepřové		333 g	
Drůbež		4,50 g	
Mléko		220 844 l	
Tejce		118 010 kusů	

tuniských (13)	-	p. Šmahlík
4 sportovní (72)	-	p. Petrůš Hlávka, D
		Štěrbová Klára
radiotechnický (12)	-	p. Václav Lac
matematický (14)	-	p. Jaroš
řidič (21)	-	p. Polášková
pěvec (20)	-	p. Hybel
divadelní (25)	-	p. Janošová
zdravotnický (30)	-	p. Jirina

I. A tř.	37 chlapců	tr. uč. Prokop Nozák
I. B tř.	- 30 děvčat	tr. uč. Vladimír Kubát
II. A tř.	22 chlapců	tr. uč. Josef Špetík
I. B tř.	29 děvčat	tr. uč. Audělna Konečná
III. tř.	17 chl. + 18 děvčat	tr. uč. Josef Dvořák
IV. tř.	10 chl. + 12 děvčat	tr. uč. Jiří Kraus
celkem	86 chl. + 89 děvčat	= 175 žáků

Figure 4.1: Examples of HerriTabNet, a new dataset created for this thesis. It consists of 600 tables in PAGE XML format, providing table bounding boxes on a page, cell bounding boxes and logical structure in a table, and text transcriptions.

Possible use cases of the dataset. As discussed above, the dataset contains bounding boxes for the tables, cells, and text lines. Also, OCR transcriptions and the cell structure, which is aligned with TSR as a primary focus. What makes the dataset multi-purpose is the possible use cases for future projects. Most intuitively, table, cell, and text line detection. Furthermore, text line clustering into cells, etc. The main quality of the chosen format is the possibility to convert the annotations to various formats, such as HTML tables for end-to-end TSR system testing, detection ground-truth formats, such as YOLO (described more in chapter 4.2), etc.

4.1 Annotation format

PAGE XML format (Page Analysis and Ground-truth Elements) [17] is an alive format used in both the scientific sector and real-world applications. Notable applications include Transkribus⁸ [8] and our own PERO-OCR⁹ [10, 11, 12]. The format's use cases in the scientific sector include Optical Character Recognition (OCR) [9], baseline detection [3], and document layout analysis [1]. Specifically for table detection and structure recognition, the format has been used in several papers and competitions [27, 5]. The format is useful to many fields thanks to flexible attributes (such as polygonal shapes made of points instead of bounding boxes) and many categories of objects (tables, images, text regions, etc.)

Specifically for the TSR, there are several tags available, such as `TableRegion` and `TableCell` with `row`, `col`, `colSpan` and `rowSpan` attributes to clearly define table structure, as seen in figure 4.2. Each visual element has its own `Coords` tag with points creating

⁸Unlock the past with Transkribus

⁹pero-ocr.fit.vutbr.cz

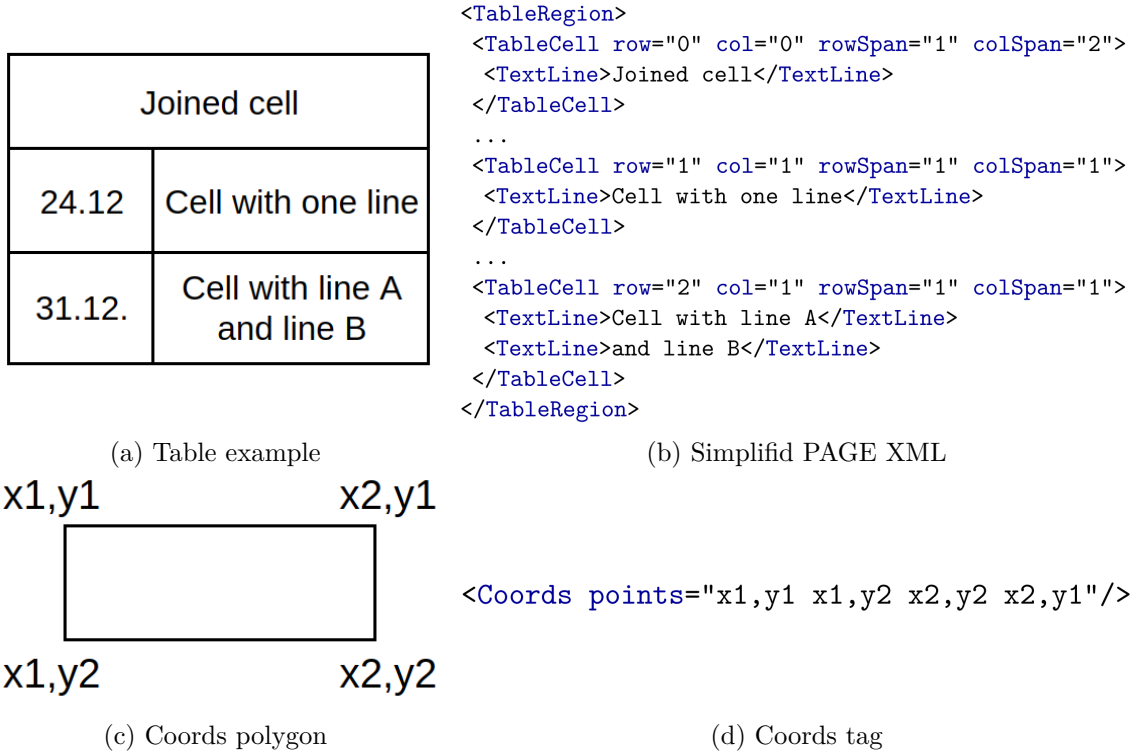


Figure 4.2: PAGE XML format (Page Analysis and Ground-truth Elements) [17] is an alive format used in both the scientific sector and real-world applications. For example, a simple table (a) is given with its PAGE XML representation (b). Each `TableCell` element has a defined logical position (and span) thanks to `row` and `col` attributes (`rowSpan` and `colSpan` respectively). Each visual element has its own `Coords` tag (d) with points creating a polygon around the object. For this thesis, we simplify all shapes to just rectangular bounding boxes (c).

a polygon around the object. For the purpose of this thesis, we simplify all shapes to rectangular bounding boxes.

4.2 Annotation process

Annotating the dataset was a multi-stage process, with the main focus being the efficiency of the annotators. Because of this, there was an annotation proposal algorithm created for each stage. Whether it was a brand new YOLO-based detector of tables, an already-existing detector of text lines, or a cell order proposition algorithm based on clustering. In every stage, there was an annotation proposition that needed to be approved or fixed by the annotators, which led to minimizing the tedious manual labor normally connected to annotation processes. An open-source framework Label studio¹⁰ was used in the entire process, providing seamless annotation proposal integrations, annotator collaboration, and work overseeing.

¹⁰labelstud.io

3 Skupen	6 I. čvrtletí %	1 II. čvrtletí %	2 III. čvrtl. %
4 1.	5 4,8	5 3,6	7 0,6
12 2.	8 6,2	10 2,3	11 0,6
13 Celkem	14 5,5	15 2,9	16 0,6

(a) Input table with detected text lines

Skupen	I. čvrtletí %	II. čvrtletí %	III. čvrtl. %
1.	4,8	3,6	0,6
2.	6,2	2,3	0,6
Celkem	5,5	2,9	0,6

(b) Table reconstruction from PAGE XML

Figure 4.3: Example from the new table dataset. In the dataset creation pipeline after detecting the table, text lines are detected and pseudo ordered. Then HTML representation is made and added to physical location information to produce PAGE XML. Table reconstruction can then be made using the cell crops.

The process started with detecting tables on pages. It continued with the detection of text lines and defining the cell structure, as well as the possibility of more text lines belonging to one cell. The last step was the text transcriptions.

Table detection proposal and approval

Table detection ground truth consists intuitively of the bounding box around the whole table. The first cca 200 tables were picked by hand, but it served as a starting dataset to train a YOLO-based table detection model based on `yolo-v8n` and trained for 100 iterations. The whole training took cca 30 minutes on a single GPU PC. The smallest model variant was chosen because of faster inference speed at the inevitable cost of accuracy. This model was then used on thousands of pages to find a subset worth giving to annotators. The model wasn't trained or evaluated on a larger dataset since it isn't the point of this thesis. It is mentioned here because it helped the table discovery for the dataset.

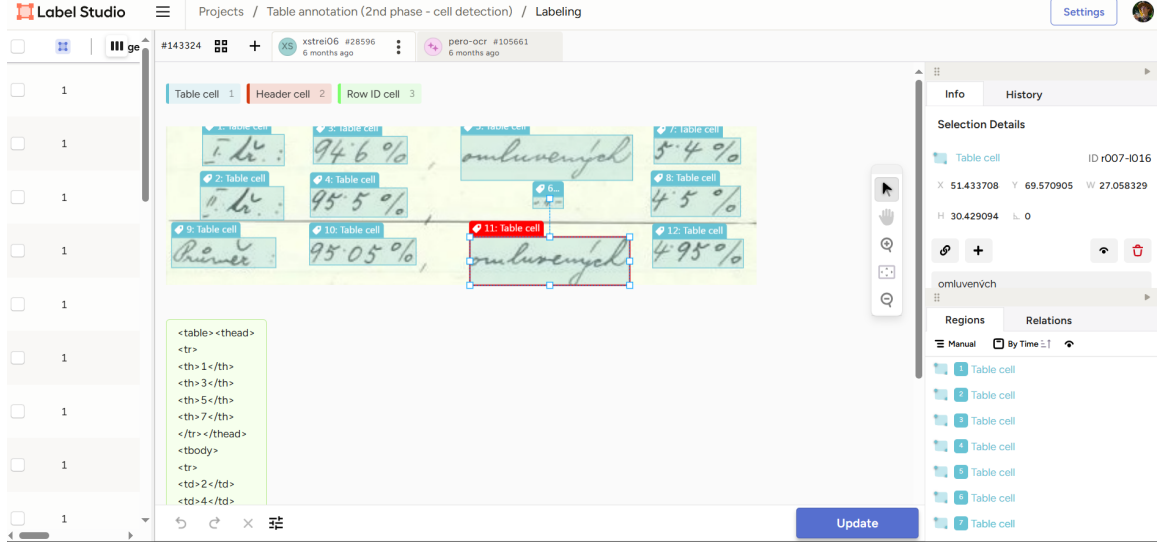


Figure 4.4: Screenshot from open-source annotation framework Label Studio web page for text line detection and structure annotation. Annotators used an external tool (table generator) to create the logical structure using HTML notation. HTML cell content corresponds to the text line IDs visible in the blue tags on top of the text line bounding boxes.

Text line detection proposal and approval, and structure annotation

Next, a text line detector was run on the detected tables. The detector was kindly provided by my colleagues in our research group. In many cases, the detector didn't split the text into cells, ignoring horizontal indentation. This is an understandable behaviour for OCR, given that its purpose is to detect text lines of any length.

To annotate table structure, a web-based table editor¹¹ was picked and used to generate HTML table representation. The logical structure was thus created using cell detection IDs as seen in figure 4.4.

After annotating this stage, the HTML representation of text line ids and the text line bounding boxes were paired, creating a final table structure in PAGE XML.

OCR transcription and word alignment

The last step was to split annotated text lines into words and get OCR transcriptions. Both these tasks were left for a brand-new universal OCR model, kindly provided by my colleague Ing. Jan Kohút. The pero-ocr framework allowed me to turn off the text line detection and load it from existing PAGE XML. So I converted Table PAGE with a logical table structure into plain PAGE XML with only `TextRegions` and `TextLines`. These files were loaded by the OCR system and without redetecting them again, the system transcribed and aligned words into the annotated text line elements. Lastly, I just created the output PAGE XML into the standardized Table PAGE XML with a logical structure in `TableRegion` and `TableCell` tags.

The OCR transcription is not the main contribution of this thesis, so the results were not approved on every piece. After reviewing a few examples, I decided it is good enough. An example can be seen in figure 4.5

¹¹www.tablesgenerator.com

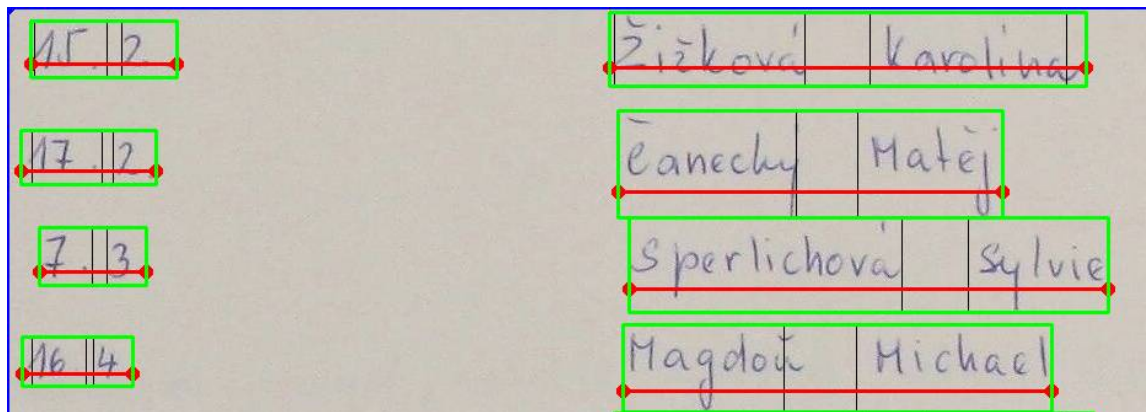


Figure 4.5: Example of automatic word alignment on premade textline detections. Text lines (green) were proposed by OCR layout model and approved / fixed by hand. Base lines (red) were placed automatically into 1/4 height of the text line. Word bounding boxes (black vertical lines) were aligned from OCR output logits.

4.3 Qualitative baseline analysis

To the best of my knowledge (and their evaluation), the best general use case model for table structure recognition was introduced in *PubTables-1M* [20] (described in section 2.3). It is a Detection Transformer model (DETR) performing detection of sub-table elements trained on the *PubTables-1M* dataset. This DETR model was chosen as a baseline for qualitative analysis of *HeritageTabNet* dataset and I downloaded it from hugging-face¹²

For each image, only detection classes „table row“ and „table column“ are displayed (in two separate images), as they provide enough information for us to have a general idea about the quality of detections. Results from *HeritageTabNet* example are available in figure 4.6. I have also added a printed table example to confirm the model works on data more resembling its training domain. The DETR model seems to not understand handwritten data in this form, and it was the case for most of the other *HeritageTabNet* samples I’ve tried. Based on these findings, quantitative analysis was not conducted.

¹²huggingface.co/microsoft/table-transformer-structure-recognition

Chapter 5

Proposed method and implementation

The codebase of this thesis is separated into three logical repositories. The `table-structure-recognition`¹ repository was created from scratch and includes *PubTables* DETR inference scripts, dataset utilities for *PubTables-1M* (described in section 2.1) and *HeritageTabNet* (described in section 4).

The *pero*-layout-organizer is the main contribution of this thesis. Its original function was training and evaluation of reading order prediction and document parts classification. I **updated** this framework to train, measure, evaluate, and infer multi-class relation prediction for table structure recognition purposes. My contributions are described more in section 5.

Finally, I **utilized** an open-source framework *pero-ocr*² with a fresh-from-the-oven universal (handwritten/printed text) OCR model trained and provided by my colleague Ing. Jan Kohút. The *pero-ocr* provided me with a solid interface framework around the OCR models and class structure capable of storing the structured OCR results and exporting to PAGE XML using the `parse_folder.py` script. I **updated** the class structure and config parameters to add word alignment, storing, and exporting to PAGE XML. Class diagram is available in figure 5.1 with updated or created parts marked with green color. All this infrastructure is based on PAGE XML standard version 2019-07-15, which allows storing both OCR and TSR structure and information. Internal distinction has been made for PAGE XML and Table PAGE XML to capture different data being stored there. Table PAGE XML is a super set thanks to the `TablePageLayout` inheriting from the original `PageLayout`.

Repository `table-structure-recognition`

I created:

- `herriage_tab_net_table_detection` with yolo inference script and script for converting yolo predictions into label-studio proposals.
- `herriage_tab_net` with every other step of the proposal creation and annotation conversion pipeline (described in section 4.2): textline detection, order proposal, result rendering, and OCR word alignment.

¹github.com/table_structure_recognition github repository

²github.com/pero-ocr

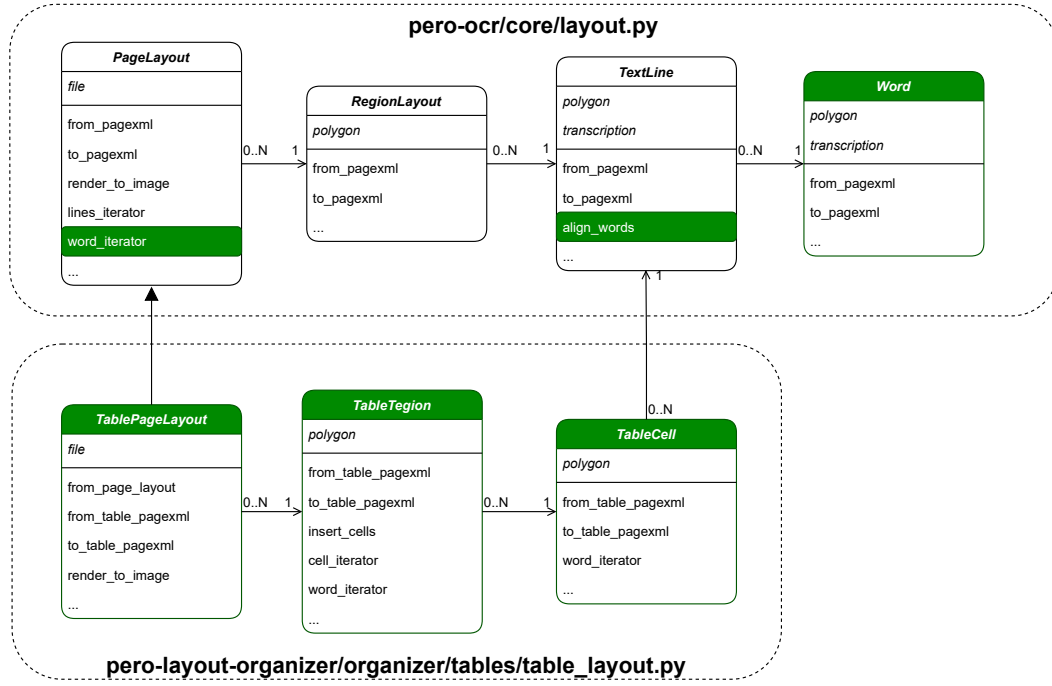


Figure 5.1: Class structure for OCR and TSR objects creation, loading and saving to PAGE XML. Classes and methods marked by green color are added for this thesis. The Table PAGE XML is only internal distinction that adds extra elements available, namely TableRegion and TableCell. All XML outputs from this framework should be valid PAGE XML according to the standard version 2019-07-15

- `pubtables_dataset_utilities` with `PubTablesConverter` responsible for creating PAGE XML with table structure representation from the dataset object detection ground-truth (individual columns, rows, etc.) and words using intersection cell creation and word assignment. This process is described more in section 4.3.
- `pubtables_detr_baseline` with `BaselineTableEngine` which provides a baseline table extraction pipeline with OCR transcription using `pero_ocr_word_engine`, table detection and table structure recognition using *PubTables* DETR [20] (described in section 2.3), and finalising it all with intersection cell creation and word assignment provided by `pubtables_dataset_utilities`. These scripts were used for qualitative baseline analysis on the *HeritageTabNet* dataset described more in section 4.3.
- `pero_ocr_word_engine`, which provides an easy interface for calling `pero-ocr` `parse_folder.py` to get text regions, text lines, and words from images.

Most of the scripts are provided with **example data** in `example_data` folders, and **default script parameters** are set to these folders. The user/reviewer can easily see input and output examples of each script right after downloading the repository (and installing libraries) and just as easily run scripts without any parameters to test the reference functionality. (An exception is made for OCR models, which are not submitted.) All the images, annotation files, and XML examples add up to 110 MB to the repository size and around the same size once again in the `.git` folder if downloaded as a git repository, which

is a considerable size and download-time overhead. For perspective, OCR models (not included in the submitted files) have around 60 MB each, so the size of the repository is in a normal range for projects like this. In the end, the benefits of having example inputs and outputs right there outweigh the minor inconveniences.

Repository `pero-layout-organizer`

In the `pero-layout-organizer` repository, I **updated** `train.py` and `model.py` with multi-class relation prediction heads and train-time evaluation. I **used** „as is“ the rest of the training and model infrastructure, including `positional_embedding.py`, `image_backbone.py`, etc. I **created** the `lmbd_wrapper.py` for preparing training data to be loaded more efficiently, the `warmup_scheduler.py`, and everything in `organizer/tables` and `organizer/tables/infer`, including:

- `table_layout.py` which expands capabilities of `pero-ocr PageLayout` with `TableRegion` and `TableCell` classes as well as loading from and exporting to PAGE XML notation in standardized table representation.
- `table_dataset.py` providing the `TableDataset` class (for training and evaluation), and rendering results
- `neighbour_matrix.py` with ground-truth creation in two modes (closest neighbour and cluster neighbours), described in sections 5.2 and 5.2 respectively.
- `rendering.py` with support for rendering graphical objects, their attributes and relations, ground-truths and predictions.
- `table_metrics.py` with F1, AUC PR and object detection metrics (described more in section 2.2)

I also created unittests in the `test` folder for continuous development testing, which saved me a lot of trouble during the process. The 24 unittests also provide examples of expected inputs and outputs for any potential user/reviewer.

5.1 Proposed method summary

The example of sub-table element detection based system described in *PubTables-1M* [20] and section 2.3 uses the OCR information at the very last stage of table extraction to construct the final structured result. This type of system can be prone to errors if some visual difference occurs. On the other hand, *ClusterTabNet* [18] described in section 2.5 suggests a framework to harness the power of an already robust OCR system and use its results as well as the input image as input for a multimodal transformer and following table structure construction. The multimodal word relation prediction approach was thus born and in this thesis, I try it myself and test this approach on a small custom dataset *HeritageTabNet* described in section 4. The whole pipeline is described in the following section and a diagram is available in figure 5.3

5.2 Word relation prediction ground-truth definitions

The transformer task is formulated as **multi-class word relation prediction**, which means for input of N word bounding boxes of shape $[N, 4]$. The ground truth is a boolean tensor of shape $[C, N, N]$, where each boolean value symbolizes if the two words are in a relation class or not. C is the number of classes. There were 3 classes in the majority of experiments in this thesis: column, row, cell. The transformer model output consists of logits for each class which is a float tensor of shape $[C, N, N]$. During training, the output logits are run through a **sigmoid function** to compute **Binary Cross-entropy loss**.

With this setting, two relation ground-truth formulations were implemented and tested. **Closest neighbour**, where words have relations only with the words of the closest neighbour cells, and **Cluster neighbours** where words have relations with all words in the same cluster class. These two formulations share training loss setup and all the tensor shapes. The difference is the semantic formulation of their ground-truths and following table structure construction.

Both ground truths are created using the `neighbour_matrix.py` functions where each input table is represented as a 2D-numpy array with indexes to the actual `TableCell` element (which has words in it). The relation matrix is then iteratively created using numpy indexing of the 2D array.

Closest neighbour

This ground-truth formulation was a first experiment with the framework of this thesis and was meant to test it to see if the training works fine. The main idea is that each word should know the best about its close surroundings, so it might be easier to train the transformer. For the cell class, all words in the cell should have a relation with each other. For the column class (row class), all words should have a relation with all words in the cell closest up and down (left and right, respectively). A visual example for the vertical class is in figure 5.2

This ground truth formulation was trained, but didn't prove to be a viable option for table structure reconstruction in the first round of tests. The accuracy comparison is available in figure 6.2

Cluster neighbours

This ground-truth formulation was heavily inspired by the *ClusterTabNet* [18] paper described in section 2.5. Word relations take the semantic meaning of „being in the same cluster“ in three classes: R_v (vertical clusters = columns), R_h (horizontal clusters = rows), and R_c (cell). Simple cell relations are symmetrical, whereas joined cells are represented in an anti-symmetrical way. For words s_1, s_2 in simple cells, the same column relations are defined like (5.1). For word j in a joined cell in the same column as both words s_1 and s_2 , relations are defined like (5.2) for relations into the word j and like (5.3) for relations from the word j . A visualization example is available in figure 5.2.

$$R_v[s_1, s_2] = R_v[s_2, s_1] = 1 \quad (5.1)$$

$$R_v[s_1, j] = R_v[s_2, j] = 1 \quad (5.2)$$

$$R_v[j, s_1] = R_v[j, s_2] = 0 \quad (5.3)$$

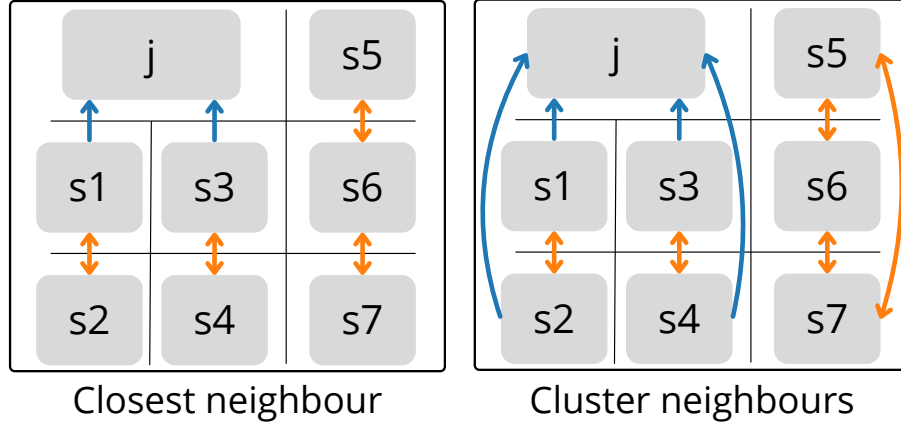


Figure 5.2: Examples of two ground truth formulation for vertical relation class R_v .

The self-loop relation aka $R[i, i]$ is semantically set to one everywhere, but it is excluded from training by a diagonal loss mask since it doesn't add any value to the training process and might even confuse the dot product similarity-based prediction heads.

5.3 Word relation prediction transformer

The transformer architecture is inspired by the *ClusterTabNet* [18] paper and *DLAFormer* [24] paper (in the way of viewing different problems as a relation prediction task). The diagram of this architecture is available in figure 5.3 and can be split into several stages.

Image & word preprocessing. In a real system pipeline, an OCR word bbox extractor would be the first step. For training purposes, word bounding boxes are loaded from the dataset. The image is preprocessed using a CNN backbone. A pretrained ResNet18[25] model from the timm library³ was used with an input image resolution of 512x512 pixels. The pixel values are normalized into 0..1 and then standardized using given **mean** and **std** parameters preset for the model. The visual features are taken from the last feature map with resolutions of 16x16 and then flattened. The CNN backbone was frozen in all training experiments. The input embedding layer is then used, and non-learnable positional embeddings are added to both the flattened image features and padded word bounding boxes to unify transformer dimensions and introduce locality.

Transformer decoder. A standard transformer decoder is used to parse both image and bounding box data embedded in the transformer embedding space. Hyper-parameters for experiments are: inner embedding dimension, number of attention heads, and decoder layers. Dimension for feed-forward decoder layers was set to 2 times the inner dimension in all experiments. Hyper-parameter variants are explored more in section 6.

Prediction heads. There is one prediction head for each class. Each one performs two parallel linear layers to create two different embeddings. Both embeddings are normalized using embeddings-wise normalization. Dot product similarity is then counted for each pair

³huggingface.co/docs/timm

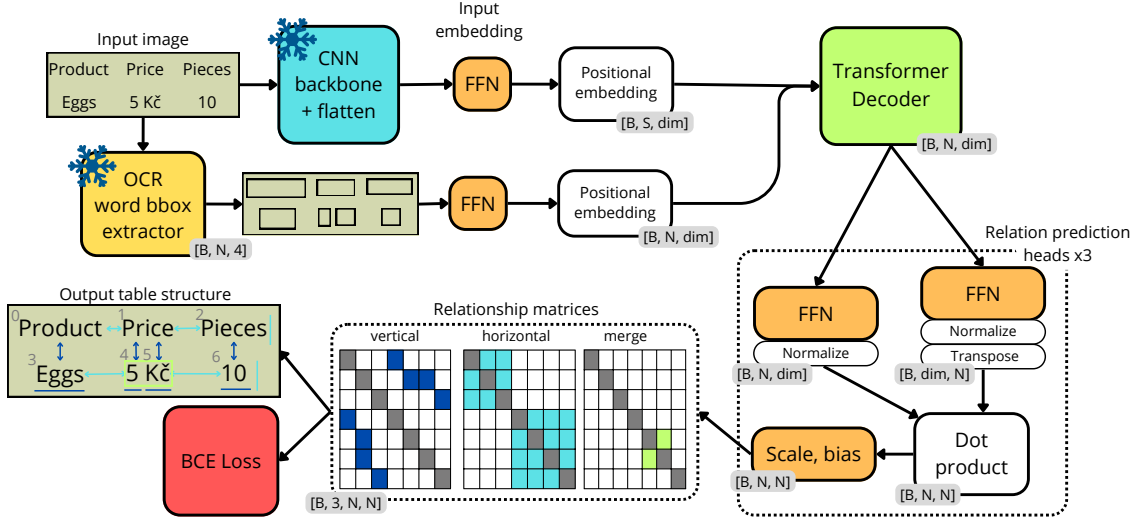


Figure 5.3: Diagram of the proposed word relation prediction TSR architecture. $\mathbf{B}$ is batch size, $\mathbf{S}$ is sequence length of flattened image backbone output, $\mathbf{dim}$ is the given dimension of transformer vector space, and $\mathbf{N}$ is the current maximal input sequence length: number of words on the input + padding.

of embeddings. This normalized dot product similarity is by definition in the $< -1, 1 >$ range, which gets sigmoided only to the $< 0.27, 0.73 >$ range, so not exactly ideal for a Binary Cross Entropy loss, which expects values either as logits (in whatever range) or as probabilities in the $< 0, 1 >$ range. Preliminary experiments also showed that similarities rarely reached bigger negative values. This can be explained mathematically since for high-dimensional embedding vectors (128-512 dimensions) it's very hard for a big amount of embedding vectors (200 in most experiments) to be all opposite to each other. The similarity values mostly resided in the $< -0.7, 1 >$ range. Class-wise scale and bias parameters were added to adjust to this mathematical constraint. The output float tensor has a shape of $[B, C, N, N]$, where B is batch size, C is classes, and N is padded words. The last two dimensions symbolize the relation values.

Custom loss aggregation. Since the tensor needs to be of coherent shape throughout dimensions, but word counts are different for each table. Padding is installed to set the N value to the preset (either cutting the input sequence or padding with zeros). The padding is masked out for loss calculation. The diagonal (self-loop relation at $R[i, i]$) of each class is also excluded since it doesn't add any value semantically and might even negatively influence the prediction heads to not change the embeddings between the two linear layers. Less change in the linear layers would mean a bigger value on the diagonal.

5.4 Table structure reconstruction

The structure reconstruction for object detection models like *PubTables* DETR [20] is relatively straightforward and summarized in section 5.4. For word relation prediction models,

the reconstruction proved to be a challenge, due to the overwhelming amount of options to test and try.

Word bounding boxes, that serve as input for the word relation prediction model, are not used for table structure reconstruction in this thesis. The reconstruction is done with the relation information alone (word bounding boxes are not taken into account anymore). The reason for this is to lower the amount of tested reconstruction parameters and to focus more on evaluating and understanding the functionality of the transformer model. This decision might worsen the accuracy on the rendered table benchmarks, but should help in a hand-written domain where words are written much more irregularly.

With the reconstruction pipeline being modular in nature, the user might be provided with an option to choose his own parameters and individual step modes to achieve the best accuracy on his target domain. Overall, this uncertainty during development can be turned into higher customizability in production.

Secondly, intersections of rows and columns are converted to cell bounding boxes, creating the basic table structure of simple cells. Spanning cells are then added via intersection to finish the predicted table structure. Individual cells are classified into column headers, projected row headers, or default cells using intersections with FA classes. Optional detected words with external OCR-like tools get assigned either to joined cells or simple cells via the maximum intersection with cell relative to the word bounding box size.

Intersection cell creation and word assignment

As mentioned earlier, 2.3 PubTables annotations and model results consist of sub-table elements. The types important for table structure reconstruction are columns, rows, and spanning cells. The following algorithm for table reconstruction is implemented in the `pubtables_dataset_utilities` mentioned in section 5 and is used for both *PubTables-1M* dataset conversion to Table PAGE XML and *PubTables* DETR inference.

The table grid with simple cells is constructed from intersections of columns and rows (sorted by center X and Y coordinate respectively). Each simple cell now has a rectangle bounding box. Spanning cells are then added to keep the detected table structure. Every word bounding box is checked for intersection first with spanning cells and then with simple cells. The word gets added to the cell with the biggest intersection relative to the word area. The final step is to split the words in the cell into multiple lines if needed. Simple DBSCAN clustering of the word center Y coordinates is performed, which proved to work perfectly as the pero-ocr system tends to represent words in line with very similar (mostly equal) Y coordinates due to the properties of the textline detector step.

Chapter 6

Experiments and results

This project was a part of the ORBIS PICTUS project ¹, which allowed me to not only have funds for creating the *HeritageTabNet* dataset (described in section 4), but also have available computational power of NVIDIA GeForce RTX 4090 with up to 24 GB memory for training and evaluation. All training was done here.

The training code came from the `pero-layout-organizer` repository, which I customized for the task of word relation prediction. The training core is pytorch² modules, loss, optimizer, and evaluation.

Training datasets and splits. All models were first trained on the *PubTables-1M* [20] (described in section 2.1). The dataset size was deemed overwhelming for real experiments, and storage capacity was limited, so randomized subsets were made due to storage capacity limitations. The split sizes are available in table 6.1. The dataset was converted into PAGE XML containing both physical structure (coordinates in the image of cells, text lines, and words) and logical structure (cell position and span in the table) using the intersection cell creation and word assignment described in section 5.4

The *HeritageTabNet* dataset was also split into training (fine-tuning), validation, and test sets. Concrete numbers are available in the table 6.1. These splits were made using stratified class sampling due to the limited amount of samples. Each class was split using a 4:1:1 ratio for train, val, and test sets respectively. The classes and their amounts were 37 tables with column spanning cells, 34 tables with row spanning cells, and top 50 tables by word count. Seeing these rather limited amounts for each class, the importance of stratified sampling is even bigger for reportability of measured results.

Model variants. Four model sizes were trained for each experiment, with the expectation of comparing model efficiency with different initial settings. See the parameter distribution for each variant in figure 6.1 and hyper-parameter settings are detailed in table 6.2

6.1 Training on PubTables-1M

Four general-purpose word relation prediction models were trained on the *PubTables-1M* subset. The data splits are described in table 6.1. All models were trained for **40 000 iter-**

¹Orbis Pictus (Reanimated book) – digitized library treasures for the creative industries

²pytorch.org

Table 6.1: Dataset splits for word relation prediction training and evaluation. Both datasets were converted to PAGE XML format including both physical structure (coordinates in the image of cells, text lines, and words) and logical structure (cell position and span in the table). The *HeritageTabNet* dataset splits were made using stratified class sampling. Each class was split using a 4:1:1 ratio for train, val, and test sets respectively. The classes were tables with column spanning cells, tables with row spanning cells, and top 50 tables by word count.

Dataset name	Train set	Val set	Test set	Origin
PubTables-1M	90 000	4 000	4 000	Generated tables
HeritageTabNet	433	100	100	Scanned & Handwritten

Table 6.2: Four model sizes were trained for each experiment, with the expectation of comparing model efficiency with different initial settings. See the parameter distribution for each variant in figure 6.1

Model variant	Uses image	Dim	Heads	Layers	lr	Parameters
Big	yes	512	8	6	8e−4	32 M
Medium	yes	256	4	4	1e−3	15 M
Small	yes	128	2	2	2e−3	12.4 M
Blind	no	128	2	2	2e−3	0.4 M

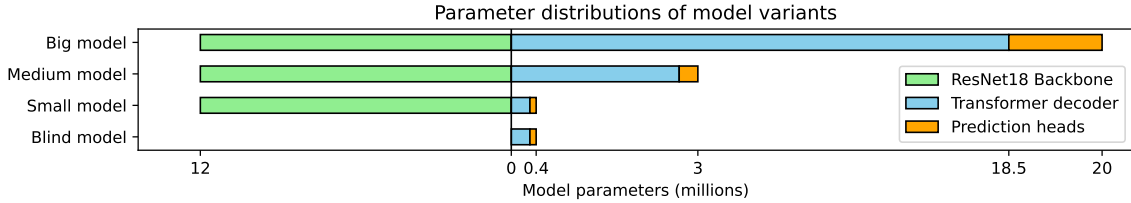


Figure 6.1: Four model sizes were trained for each experiment. The figure shows the parameter distributions in model parts. ResNet18 backbone was frozen during training. The hyper-parameter settings are detailed in table 6.2

ations and **batch size 128**. A **linear warmup** scheduler was used with 2 000 iterations (for blind, small, and medium model sizes) and 4 000 iterations (for the big model).

Two optimization strategies. Two types of optimization strategies were tested. In the basic experiment, all model parameters (except the frozen ResNet18 backbone) get trained using a central **AdamW** optimizer with the learning rate set appropriately to the size of the current transformer decoder. A specialized **AdamW** optimizer was added in the „head optimizer“ experiments. This optimizer reacted to the same loss values as the main one, but optimized only the head linear layers and scale + bias model parameters. It had a much bigger learning rate, set to 0.1. This was done to test a theory which stated that the head parameters get minimal weight updates during training due to the main optimizer learning rate being set for the transformer decoder. The theory’s origin is at the poor separability of tested outputs described more in section 6.3 and an example is given in figure 6.6.

Results of these experiments were only incremental improvements on measured accuracies DAR F1 and ARI, not game changers. Results of this experiment are discussed in

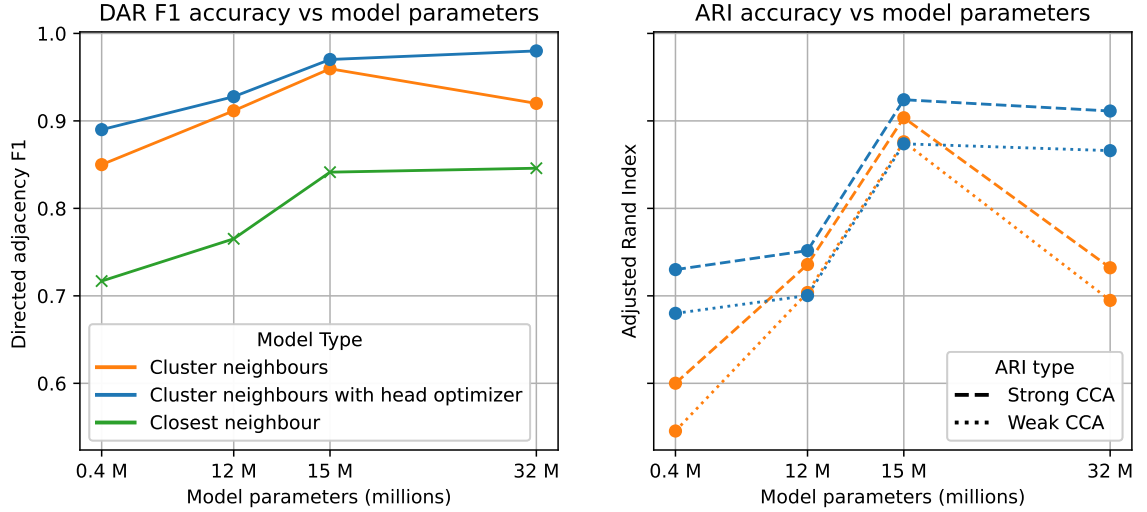


Figure 6.2: Accuracies of word relation prediction models trained on *PubTables-1M* train set and measured on *PubTables-1M* val set. Predicted relation logits were thresholded with 0 threshold to create boolean predictions required by both DAR F1 and ARI. The **DAR F1 (Directed Adjacency Relation)** accuracy measures the model output and ground truth directly. The **ARI (Adjusted Rand Index)** measures the model output capability to split input elements into clusters same as are in the ground truth. Clusters for both ground truth and prediction are created with strong and weak connected component analysis (CCA). Strong CCA includes only simple cells (clusters being columns and rows). Weak CCA views the relations as undirected and thus is connecting joined cells with clusters (creating groups of columns/rows connected by the joined cell), simple visualization is in figure 2.4. Both ARI for strong and weak CCA are added here, because together they more closely resamples the use case in reconstruction.

figure 6.2. Overall, it looks like both strong and weak ARI behave similarly to each other and to DAR F1 accuracy. The difference is that ARI is always lower than F1, meaning the prediction errors important for CCA are penalized more.

6.2 Fine-tuning on HeritageTabNet

Dataset. A mixed training dataset was created to prevent overfitting on the dataset. Training data was composed of 400 samples from *HeritageTabNet* (whole train set) and randomly selected 1 000 samples from *PubTables-1M* train set. Validation loss and accuracy were measured on both *HeritageTabNet* val set, *PubTables-1M* val set, and the mixed train set.

Hyper-parameters. Learning rate of the main optimizer (training the decoder) was decreased to around $5e-4$, and batch size was also decreased from 128 to 32 samples. Head optimizer learning rate was left stable at 0.1, which basically made sure that the decoder does not overfit data and that the training is happening mostly at the prediction heads.

Results. To compare results of fine-tuned vs the original models, all variants were tested on both *HeritageTabNet* test set and *PubTables-1M* test set. Comparison results are avail-

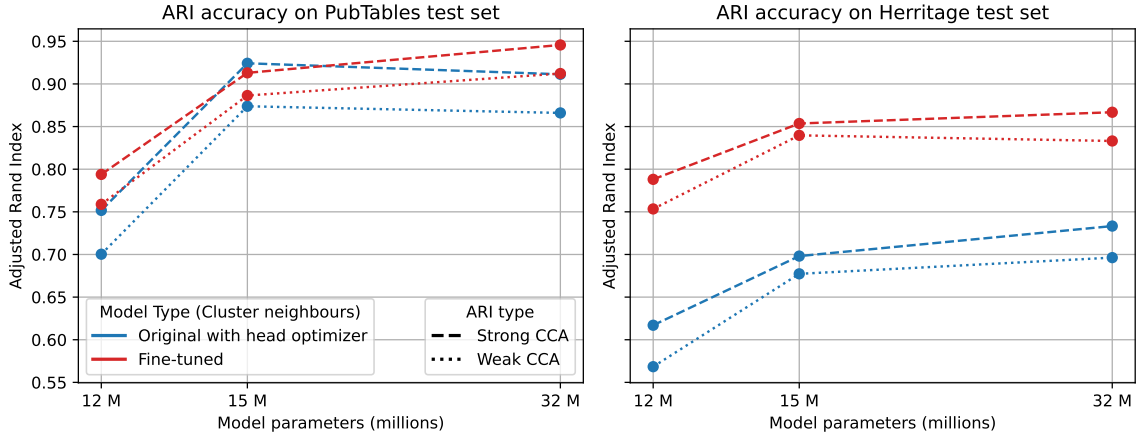


Figure 6.3: Accuracies of word relation prediction models all trained on the train set from *PubTables-1M* [20]. „Fine-tuned“ models were then fine-tuned on a mix of *HeritageTabNet* train set and *PubTables-1M* subset. Accuracies are measured on test sets of both datasets. **ARI strong** measures model capability of creating coherent clusters of simple cells. **ARI weak** measures the same and adds capability of correctly locating joined cells. Fine-tuning not only boosted accuracy on *HeritageTabNet* greatly, but more importantly did not damage the accuracies on *PubTables-1M*.

able in figure 6.3 and show successful fine-tuning to previously unseen dataset without losing accuracy on a general purpose dataset.

6.3 Table structure reconstruction with clustering

Having the model imitate ground truth successfully is one thing. Reconstructing the actual table is another. The *ClusterTabNet* [18] paper has the same word relation prediction ground truth (clustering with uni-directional and bi-directional relations), and it argues for thresholding prediction probabilities and then running connected component analysis. This didn’t work in my case due to reasons illustrated later in this section. I developed a new algorithm that utilizes the prediction probabilities before thresholding and clusters words and cells using clustering algorithms (mostly DBSCAN). This allowed the pipeline to evolve in a new direction and achieve comparable results.

Thresholded reconstruction. The suggested pipeline in the *ClusterTabNet* [18] consists of thresholding predicted relations, followed by a strongly connected component analysis³ to produce clusters of cells, columns, and rows without spanning cells (simple cell structure). The next step is to use the created column and row clusters and apply a weakly connected component analysis⁴ to find which spanning cells belong to which columns and/or rows.

For this thresholding approach to work, model has to clearly and reliably split the prediction values into positive and negative predictions. The threshold can then be set for the given model based on tests on the validation set. In my case, things were a bit more

³In a strongly connected component, all nodes have a path between each other, edges are treated as directed

⁴In a weakly connected component, all nodes have a path between each other, but edges are treated as undirected

difficult. Train-time validation and test accuracy showed promising results with thresholded F1 and multi-threshold AUC PR and AUC ROC values over 99 % . Later evaluation was conducted with the Adjusted Rand Index (described in section 2.2) to evaluate the clustering capabilities of the outputs. This evaluation showed much less optimistic results because of high sensitivity to the threshold. An illustrative example of the problem is available in figure 6.6.

Fuzzy reconstruction. After observing the uncertainty and high sensitivity to threshold selection in the thresholding approach, I developed a fuzzy reconstruction method that leverages the model’s prediction probabilities before any hard thresholding is applied. Density-based clustering algorithms such as density-based DBSCAN⁵ iteratively expand clusters by connecting densely packed points while identifying points in low-density regions—such as uncertain or inconsistent model outputs—as outliers. This inherent property of density-based clustering algorithms has the potential to improve inference clustering by utilizing predictions in the $< 0, 1 >$ range. The fuzzy table reconstruction algorithm is implemented in the `pero-layout-organizer` repository in `table_constructor.py`

Summary of the fuzzy reconstruction algorithm is provided here, and visualization is available in figure 6.4

1. **Convert logits to probabilities** with min-max normalization.
2. **Cluster the „same cell“ class to create cells** and average inter-cell word relations in the remaining relation classes to create cell relations.
3. **Ignore joined cells for now.** Ignore uni-directional cell relations using $R_i^{min} = \min(R_i, R_i^T)$, where R_i is the relation matrix of probabilities (for the remaining column and row matrices).
4. **Cluster columns and rows.** The joined cells are marked as outliers due to minimized uni-directional relations in the R_i^{min} matrices.
5. **Add joined cells to 0..N rows and columns.** The clusters→outliers relation matrix is acquired using $R_i^{max} = \max(R_i, R_i^T)$ and then averaging inter-cluster relations. Outliers (joined cells) get added to 0..N clusters based on the thresholded clusters→outliers matrix. Zero or all positive thresholds means a new cluster (column or row) is created with only the joined cell. These clusters are now non-disjunctive, since joined cells belong to multiple columns and/or rows.
6. **Combine column and row clusters to create table structure.** Simple cells are added first and joined cells are then added to non-occupied space.

After trying a few clustering algorithm alternatives for each clustering step, the DBSCAN turned out to work the best, due to its ability to choose outliers and its threshold stability. Quantitative analysis of different algorithms is not included in this thesis, but qualitative tests during development included

- **HDBSCAN** which didn’t perform as well for any chosen threshold.
- **hierarchical linkage clustering** which by definition doesn’t choose outliers but more importantly also showed threshold instability for different amount of words on input.

⁵scikit-learn.org/.../sklearn.cluster.DBSCAN

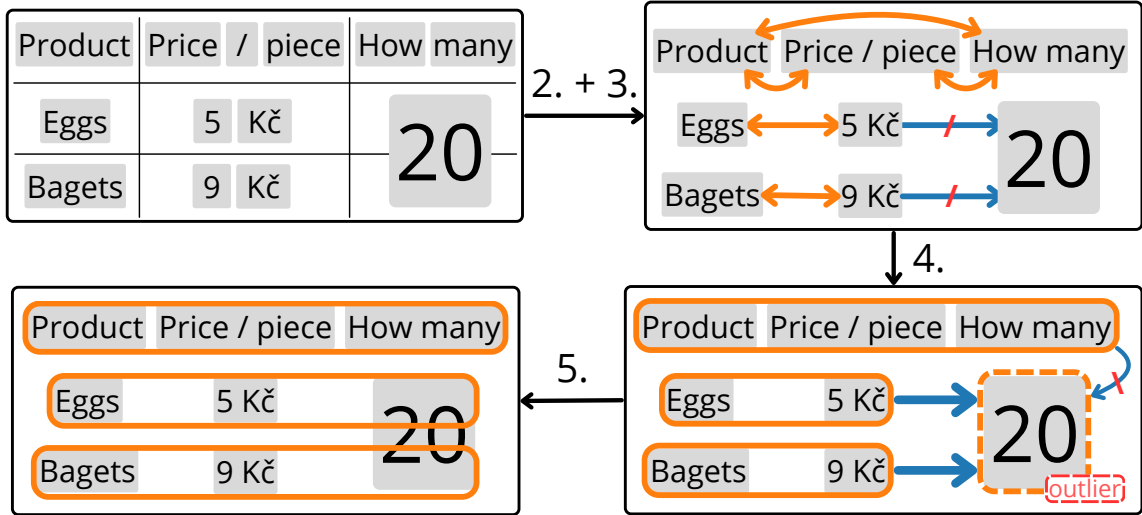


Figure 6.4: Example visualization of fuzzy reconstruction from word relation prediction model to table structure. This example shows only row clustering for clarity. The algorithm goes like this: **1.** Convert logits to probabilities. **2.** Cluster the „same cell“ class to create cells. **3.** Ignore joined cells for now. **4.** Cluster columns and rows. **5.** Add joined cells to 0..N rows and columns. **6.** Combine column and row clusters to create the table structure

Table reconstruction results. The measured results are available in figure 6.5, which clearly shows reference model superiority on the large *PubTables* dataset. The figure also shows incremental improvement when comparing the original and fine-tuned models, which would suggest that the fine-tuning process helped the model better understand more complex data samples.

Results on the *HeritageTabNet* dataset aren’t as good as expected, but considering the original model had much lower accuracies (and had to be cut from the plot for stretching the lower limit too much), at least the fine-tuning is proven to work.

Four examples with detection accuracies are available in the appendix. Figure B.1 shows an example of two tables from *PubTables* and their evaluation. Figures B.2 and B.3 show samples from *HeritageTabNet* test set with varying accuracies. Looking at the results, the first problem that should be solved is word merging into cells, since this affects all later stages of the pipeline and most of the viewed mistakes originated from this. The DBSCAN algorithm with 0.2 as the epsilon parameter showed the best results of tried options, but still is less than ideal.

6.4 Discussion

While it is clear that the fine-tuning helped achieve better results, the overall results are not as good as expected. Either the reconstruction algorithm needs more work, or models are weaker than thought based on direct output accuracies, which would mean some different accuracies are needed.

The qualitative analysis here is a difficult task. There isn’t a clear distinction of model error vs. reconstruction error when reviewing samples one by one, so the developer is left guessing.

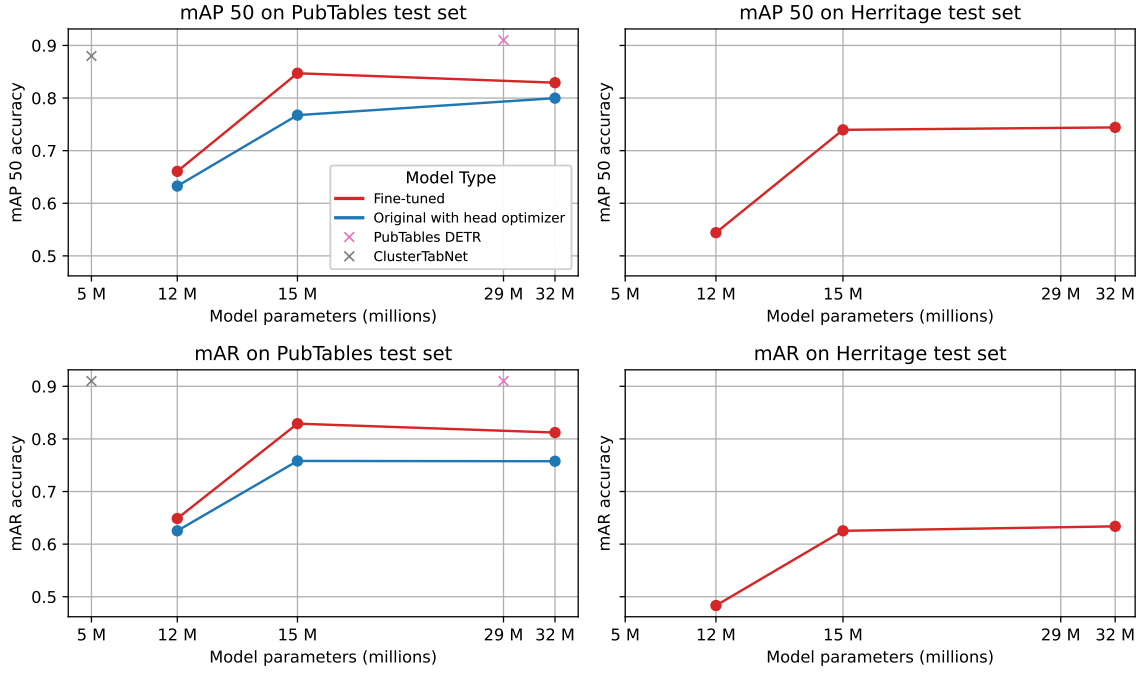
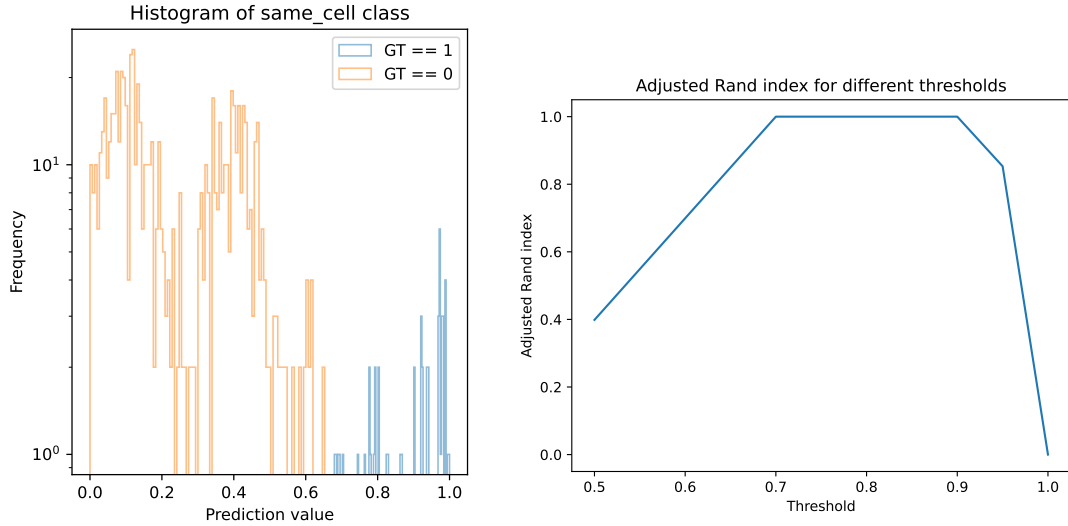


Figure 6.5: Mean Average Precision and Mean Average Recall results original and fine-tuned word relation prediction models measured on *PubTables* [20] test set (left) and *HeritageTabNet* test set (right). Two reference results are added (*PubTables* DETR and *ClusterTabNet* [18]). The bounding boxes for precision and recall measuring (columns and rows) get built from individual word bounding boxes using the word relation prediction model output and fuzzy reconstruction algorithm. The results show that even my best models have lower accuracy than reference models. There is a visible improvement with each fine-tuned model so there is definitely space for improvement.

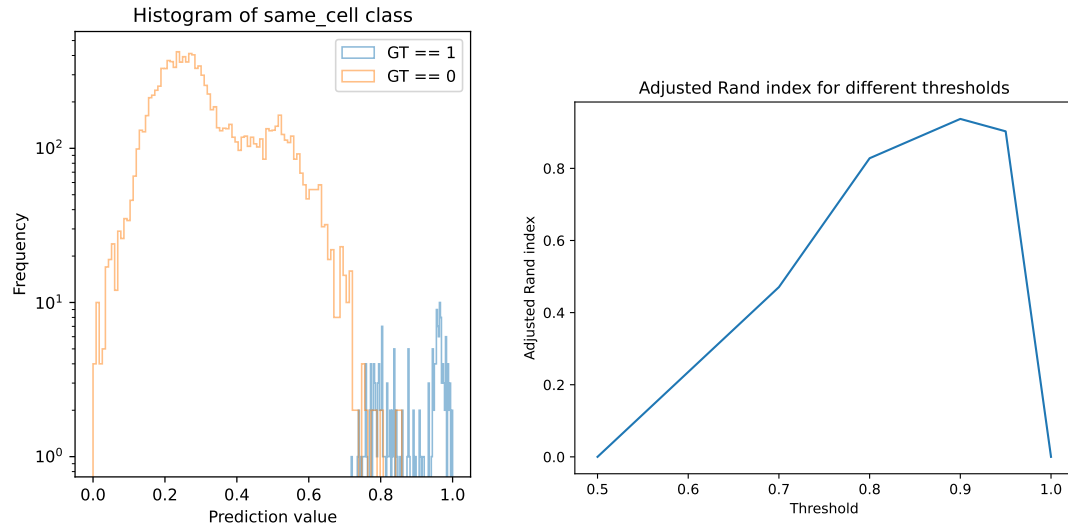
One thing is sure, for future development both the model (and its ground-truth) must evolve with the post-processing together.

The most mistakes came from two places. Over-merging words into cells, which looked like a model prediction issue, since it occurred mostly in some weird places that the model just wasn't ready for. Another frequent issue was incorrect joined cell assignment. Training ground-truth update might be needed with classification into joined cell or simple cell to clarify the reconstruction process.

All experiments in this thesis were trained with 200 input words max. In the datasets it was enough for about 60 % of tables. In the overflowing tables, only the first 200 words were left to create somewhat concise table shape. If a bigger word number training couldn't be possible, but bigger tables were needed to be processed, a random word selection might be applied to create the table structure from a subset of words. Not selected words should be then added to the created table using some form of intersection assignment.



(a) Nicely separated predictions



(b) Poorly separated prediction

Figure 6.6: Illustration of problematic separation of word relation predictions (left side) and its influence on the ARI (Adjusted Rand Index) clustering score and sensitive threshold results (right side). The prediction value is min-max normalized output logits. This can be done without sigmoid because they are mathematically in a fixed range due to the nature of predictio heads. Output logits are scaled normalized dot-product so they are in a fixed range.

Chapter 7

Conclusion

Main goals of this thesis were to implement and train a new approach to Table Structure Recognition and test it on a large generated dataset and the custom *HeritageTabNet* dataset of handwritten historical tables created for this thesis. The goals were met. The created TSR system has results comparable with previous implementations. 0.85 mean Average Precision and 0.83 mean Average Recall. A custom-made post-processing algorithm called fuzzy table reconstruction was created and tested to achieve the mentioned results.

The new *HeritageTabNet* contains 633 annotated tables in a standardized PAGE XML format, ideal for a few different tasks including TSR, table detection, text line and word detection, and handwritten Optical Character Recognition. It was annotated in a manual correction of automatic proposals framework. TSR results in this thesis can serve as a baseline for future development.

Future development might revisit ground truth creation and consider adding some information to clarify the post-processing stage. And, as always, more training experiments.

This thesis allowed me to learn how to work with large datasets and try a literature-backed approach in my own implementation. It also created trust issues for accuracy metrics, since choosing informative metrics for different parts of the pipeline and/or qualities of predictions proved to be a challenge.

Bibliography

- [1] BINMAKHASHEN, G. M. and MAHMOUD, S. A. Document layout analysis: a comprehensive survey. *ACM Computing Surveys (CSUR)*. ACM New York, NY, USA, 2019, vol. 52, no. 6, p. 1–36.
- [2] CARION, N.; MASSA, F.; SYNNAEVE, G.; USUNIER, N.; KIRILLOV, A. et al. End-to-end object detection with transformers. In: Springer. *European conference on computer vision*. 2020, p. 213–229.
- [3] DIEM, M.; KLEBER, F.; FIEL, S.; GRÜNING, T. and GATOS, B. CBAD: ICDAR2017 competition on baseline detection. In: IEEE. *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*. 2017, vol. 1, p. 1355–1360.
- [4] GAO, L.; HUANG, Y.; DÉJEAN, H.; MEUNIER, J.-L.; YAN, Q. et al. ICDAR 2019 competition on table detection and recognition (cTDaR). In: IEEE. *2019 International Conference on Document Analysis and Recognition (ICDAR)*. 2019, p. 1510–1515.
- [5] GAO, L.; YI, X.; JIANG, Z.; HAO, L. and TANG, Z. ICDAR2017 competition on page object detection. In: IEEE. *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*. 2017, vol. 1, p. 1417–1422.
- [6] GÖBEL, M.; HASSAN, T.; ORO, E. and ORSI, G. A methodology for evaluating algorithms for table understanding in PDF documents. In: *Proceedings of the 2012 ACM symposium on Document engineering*. 2012, p. 45–48.
- [7] HE, K.; ZHANG, X.; REN, S. and SUN, J. Deep residual learning for image recognition. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, p. 770–778.
- [8] KAHLE, P.; COLUTTO, S.; HACKL, G. and MÜHLBERGER, G. Transkribus-a service platform for transcription, recognition and retrieval of historical documents. In: IEEE. *2017 14th iapr international conference on document analysis and recognition (icdar)*. 2017, vol. 4, p. 19–24.
- [9] KIŠŠ, M.; HRADIŠ, M. and KODYM, O. Brno mobile OCR dataset. In: IEEE. *2019 International Conference on Document Analysis and Recognition (ICDAR)*. 2019, p. 1352–1357.
- [10] KIŠŠ, M.; BENEŠ, K. and HRADIŠ, M. AT-ST: self-training adaptation strategy for OCR in domains with limited transcriptions. In: Springer. *Document Analysis and*

Recognition-ICDAR 2021: 16th International Conference, Lausanne, Switzerland, September 5–10, 2021, Proceedings, Part IV 16. 2021, p. 463–477.

- [11] KODYM, O. and HRADIŠ, M. Page layout analysis system for unconstrained historic documents. In: Springer. *Document Analysis and Recognition-ICDAR 2021: 16th International Conference, Lausanne, Switzerland, September 5–10, 2021, Proceedings, Part II 16.* 2021, p. 492–506.
- [12] KOHÚT, J. and HRADIŠ, M. TS-Net: OCR trained to switch between text transcription styles. In: Springer. *Document Analysis and Recognition-ICDAR 2021: 16th International Conference, Lausanne, Switzerland, September 5–10, 2021, Proceedings, Part IV 16.* 2021, p. 478–493.
- [13] KUDALE, D.; KASUBA, B. V.; SUBRAMANIAN, V.; CHAUDHURI, P. and RAMAKRISHNAN, G. Sprint: Script-agnostic Structure Recognition in Tables. In: Springer. *International Conference on Document Analysis and Recognition.* 2024, p. 350–367.
- [14] LU, N.; YU, W.; QI, X.; CHEN, Y.; GONG, P. et al. Master: Multi-aspect non-local network for scene text recognition. *Pattern Recognition.* Elsevier, 2021, vol. 117, p. 107980.
- [15] LYSAK, M.; NASSAR, A.; LIVATHINOS, N.; AUER, C. and STAAR, P. Optimized Table Tokenization for Table Structure Recognition. In: Springer. *International Conference on Document Analysis and Recognition.* 2023, p. 37–50.
- [16] PIWOWARSKI, B. L. *NEURAL NETWORKS FOR AUTOMATIC TABLE RECOGNITION.* Brno, CZ, 2022. Diplomová práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Available at: <https://www.vut.cz/studenti/zav-prace/detail/145454>.
- [17] PLETSCHACHER, S. and ANTONACIOPOULOS, A. The page (page analysis and ground-truth elements) format framework. In: IEEE. *2010 20th International Conference on Pattern Recognition.* 2010, p. 257–260.
- [18] POLEWCZYK, M. and SPINACI, M. ClusterTabNet: Supervised clustering method for table detection and table structure recognition. In: Springer. *International Conference on Document Analysis and Recognition.* 2024, p. 334–349.
- [19] REN, S.; HE, K.; GIRSHICK, R. and SUN, J. Faster R-CNN: Towards real-time object detection with region proposal networks. *IEEE transactions on pattern analysis and machine intelligence.* IEEE, 2016, vol. 39, no. 6, p. 1137–1149.
- [20] SMOCK, B.; PESALA, R. and ABRAHAM, R. PubTables-1M: Towards comprehensive table extraction from unstructured documents. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition.* 2022, p. 4634–4642.
- [21] SMOCK, B.; PESALA, R. and ABRAHAM, R. GriTS: Grid table similarity metric for table structure recognition. In: Springer. *International Conference on Document Analysis and Recognition.* 2023, p. 535–549.
- [22] VASWANI, A. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

- [23] VLACH, V. *Automatické rozpoznávání hudebního zápisu pomocí neuronových sítí*. Brno, CZ, 2023. Bakalářská práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Available at: <https://www.fit.vut.cz/study/thesis/7848/>.
- [24] WANG, J.; HU, K. and HUO, Q. DLAFormer: An End-to-End Transformer For Document Layout Analysis. In: Springer. *International Conference on Document Analysis and Recognition*. 2024, p. 40–57.
- [25] WIGHTMAN, R.; TOUVRON, H. and JÉGOU, H. Resnet strikes back: An improved training procedure in timm. *ArXiv preprint arXiv:2110.00476*, 2021.
- [26] ZHONG, X.; SHAFIEIBAVANI, E. and JIMENO YEPES, A. Image-based table recognition: data, model, and evaluation. In: Springer. *European conference on computer vision*. 2020, p. 564–580.
- [27] ZIOMEK, J. and MIDDLETON, S. E. GloSAT historical measurement table dataset: enhanced table structure recognition annotation for downstream historical data rescue. In: *Proceedings of the 6th International Workshop on Historical Document Imaging and Processing*. 2021, p. 49–54.

Appendix A

Attachments structure

The complete attachments for this thesis are available at nextcloud¹ and have the following structure. A detailed description of each code repository is available in chapter 5.

- **table_structure_recognition** repository, which is my own creation. Live version is available on GitHub²
- **pero-layout-organizer** repository which I partly utilized but created all the TSR related implementation.
- **pero-ocr** repository which I updated with a few changes. Live version is available on GitHub³.
- **herritagetabnet_dataset** with final dataset of images (table crops), corresponding annotations in PAGE XML, and split information
- **trained_models** with config files
- **docs** with L^AT_EX source codes and other material for the thesis text.
- **video**

¹<https://nextcloud.fit.vutbr.cz/s/RM7QiBy9y2w4Tbs>

²github.com/vlachvojta/table_structure_recognition

³github.com/DCGM/pero-ocr

Appendix B

Table extraction examples

Correct

predictions						ground truths				
1 cancer type Invasive DCIS 2 Total no. of tumours 101 102 3 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+) 4 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+)					vertical	1 cancer type Invasive DCIS 2 Total no. of tumours 101 102 3 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+) 4 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+)				
1 cancer type Invasive DCIS 2 Total no. of tumours 101 102 3 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+) 4 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+)					horizontal	1 cancer type Invasive DCIS 2 Total no. of tumours 101 102 3 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+) 4 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+)				
1 cancer type Invasive DCIS 2 Total no. of tumours 101 102 3 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+) 4 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+)					spanning	1 cancer type Invasive DCIS 2 Total no. of tumours 101 102 3 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+) 4 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+)				
1 cancer type Invasive DCIS 2 Total no. of tumours 101 102 3 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+) 4 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+)					cells	1 cancer type Invasive DCIS 2 Total no. of tumours 101 102 3 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+) 4 IBCP-3 IHC score Negative (0) Weak/moderate (1+) Strong (2+)				

Over-joined cells

predictions						ground truths				
1 Plasma Glucose (mmol/mol) Serum Insulin (μU/ml) Insulin to Glucose Ratio^A Serum Adiponectin (ng/ml) Serum Leptin (ng/ml) Adiponectin to Leptin Ratio^A 2 GDM Not GDM Treatment Group Routine Care Group Control Group 3 GDM Not GDM Treatment Group Routine Care Group Control Group 4 GDM Not GDM Treatment Group Routine Care Group Control Group					vertical	1 Plasma Glucose (mmol/mol) Serum Insulin (μU/ml) Insulin to Glucose Ratio^A Serum Adiponectin (ng/ml) Serum Leptin (ng/ml) Adiponectin to Leptin Ratio^A 2 GDM Not GDM Treatment Group Routine Care Group Control Group 3 GDM Not GDM Treatment Group Routine Care Group Control Group 4 GDM Not GDM Treatment Group Routine Care Group Control Group				
1 Plasma Glucose (mmol/mol) Serum Insulin (μU/ml) Insulin to Glucose Ratio^A Serum Adiponectin (ng/ml) Serum Leptin (ng/ml) Adiponectin to Leptin Ratio^A 2 GDM Not GDM Treatment Group Routine Care Group Control Group 3 GDM Not GDM Treatment Group Routine Care Group Control Group 4 GDM Not GDM Treatment Group Routine Care Group Control Group					horizontal	1 Plasma Glucose (mmol/mol) Serum Insulin (μU/ml) Insulin to Glucose Ratio^A Serum Adiponectin (ng/ml) Serum Leptin (ng/ml) Adiponectin to Leptin Ratio^A 2 GDM Not GDM Treatment Group Routine Care Group Control Group 3 GDM Not GDM Treatment Group Routine Care Group Control Group 4 GDM Not GDM Treatment Group Routine Care Group Control Group				
1 Plasma Glucose (mmol/mol) Serum Insulin (μU/ml) Insulin to Glucose Ratio^A Serum Adiponectin (ng/ml) Serum Leptin (ng/ml) Adiponectin to Leptin Ratio^A 2 GDM Not GDM Treatment Group Routine Care Group Control Group 3 GDM Not GDM Treatment Group Routine Care Group Control Group 4 GDM Not GDM Treatment Group Routine Care Group Control Group					spanning	1 Plasma Glucose (mmol/mol) Serum Insulin (μU/ml) Insulin to Glucose Ratio^A Serum Adiponectin (ng/ml) Serum Leptin (ng/ml) Adiponectin to Leptin Ratio^A 2 GDM Not GDM Treatment Group Routine Care Group Control Group 3 GDM Not GDM Treatment Group Routine Care Group Control Group 4 GDM Not GDM Treatment Group Routine Care Group Control Group				
1 Plasma Glucose (mmol/mol) Serum Insulin (μU/ml) Insulin to Glucose Ratio^A Serum Adiponectin (ng/ml) Serum Leptin (ng/ml) Adiponectin to Leptin Ratio^A 2 GDM Not GDM Treatment Group Routine Care Group Control Group 3 GDM Not GDM Treatment Group Routine Care Group Control Group 4 GDM Not GDM Treatment Group Routine Care Group Control Group					cells	1 Plasma Glucose (mmol/mol) Serum Insulin (μU/ml) Insulin to Glucose Ratio^A Serum Adiponectin (ng/ml) Serum Leptin (ng/ml) Adiponectin to Leptin Ratio^A 2 GDM Not GDM Treatment Group Routine Care Group Control Group 3 GDM Not GDM Treatment Group Routine Care Group Control Group 4 GDM Not GDM Treatment Group Routine Care Group Control Group				

Figure B.1: Examples of table reconstruction result (left) with ground truth (right). The upper is correctly extracted, the lower has over-joined cells and therefore messed up later stages. mAP₅₀: 0.57, mAR: 0.67

predictions						ground truths					
Drůbek:	Předpis:	dodětky:	Druh:	Předpis:	dodětky:	Drůbek:	Předpis:	dodětky:	Druh:	Předpis:	dodětky:
žemice	110	108.50	Hovězí/dobýtek	13.887	25.368	žemice	110	108.50	Hovězí/dobýtek	13.887	25.368
žito	217	277	Lepravy/dobýtek	8.315	9.924	žito	217	277	Lepravy/dobýtek	8.315	9.924
ječmen	4.17	32.85	Mleko		62.880	ječmen	4.17	32.85	Mleko		62.880
over			voje	27.180	26.120	over			voje	27.180	26.120
nač			hrášek	32	30	nač			hrášek	32	30
kovice						kovice					
len						len					
brambory	592	685				brambory	592	685			
= nedodáno předáno						= nedodáno předáno					
Drůbek:	Předpis:	dodětky:	Druh:	Předpis:	dodětky:	Drůbek:	Předpis:	dodětky:	Druh:	Předpis:	dodětky:
žemice	110	108.50	Hovězí/dobýtek	13.887	25.368	žemice	110	108.50	Hovězí/dobýtek	13.887	25.368
žito	217	277	Lepravy/dobýtek	8.315	9.924	žito	217	277	Lepravy/dobýtek	8.315	9.924
ječmen	4.17	32.85	Mleko		62.880	ječmen	4.17	32.85	Mleko		62.880
over			voje	27.180	26.120	over			voje	27.180	26.120
nač			hrášek	32	30	nač			hrášek	32	30
kovice						kovice					
len						len					
brambory	592	685				brambory	592	685			
= nedodáno předáno						= nedodáno předáno					
Drůbek:	Předpis:	dodětky:	Druh:	Předpis:	dodětky:	Drůbek:	Předpis:	dodětky:	Druh:	Předpis:	dodětky:
žemice	110	108.50	Hovězí/dobýtek	13.887	25.368	žemice	110	108.50	Hovězí/dobýtek	13.887	25.368
žito	217	277	Lepravy/dobýtek	8.315	9.924	žito	217	277	Lepravy/dobýtek	8.315	9.924
ječmen	4.17	32.85	Mleko		62.880	ječmen	4.17	32.85	Mleko		62.880
over			voje	27.180	26.120	over			voje	27.180	26.120
nač			hrášek	32	30	nač			hrášek	32	30
kovice						kovice					
len						len					
brambory	592	685				brambory	592	685			
= nedodáno předáno						= nedodáno předáno					
Drůbek:	Předpis:	dodětky:	Druh:	Předpis:	dodětky:	Drůbek:	Předpis:	dodětky:	Druh:	Předpis:	dodětky:
žemice	110	108.50	Hovězí/dobýtek	13.887	25.368	žemice	110	108.50	Hovězí/dobýtek	13.887	25.368
žito	217	277	Lepravy/dobýtek	8.315	9.924	žito	217	277	Lepravy/dobýtek	8.315	9.924
ječmen	4.17	32.85	Mleko		62.880	ječmen	4.17	32.85	Mleko		62.880
over			voje	27.180	26.120	over			voje	27.180	26.120
nač			hrášek	32	30	nač			hrášek	32	30
kovice						kovice					
len						len					
brambory	592	685				brambory	592	685			
= nedodáno předáno						= nedodáno předáno					
Drůbek:	Předpis:	dodětky:	Druh:	Předpis:	dodětky:	Drůbek:	Předpis:	dodětky:	Druh:	Předpis:	dodětky:
žemice	110	108.50	Hovězí/dobýtek	13.887	25.368	žemice	110	108.50	Hovězí/dobýtek	13.887	25.368
žito	217	277	Lepravy/dobýtek	8.315	9.924	žito	217	277	Lepravy/dobýtek	8.315	9.924
ječmen	4.17	32.85	Mleko		62.880	ječmen	4.17	32.85	Mleko		62.880
over			voje	27.180	26.120	over			voje	27.180	26.120
nač			hrášek	32	30	nač			hrášek	32	30
kovice						kovice					
len						len					
brambory	592	685				brambory	592	685			
= nedodáno předáno						= nedodáno předáno					

Figure B.2: Example of table reconstruction result (left) with ground truth (right).
mAP_50: 0.704, mAR: 0.735

[illegible]

Figure B.3: Example of table reconstruction result (left) with ground truth (right).
mAP_50: 0.76, mAR: 0.64